

Universidad Nacional Autónoma de México
Facultad de Ciencias

Modelacion Basada en Agentes
Semestre: 2026-1

Practica 1

García Ponce José Camilo 319210536

- Parte 1

- A. ¿Qué es la modelación basada en agentes?

- Es una metodología para el estudio de sistemas complejos mediante la colección de agentes que interactúan con el entorno a través del tiempo.

- Los agentes tienen reglas y comportamientos.

- B. ¿Qué es el enfoque bottom-up?

- Cuando se consideran a los agentes de manera individual y se van viendo las interacciones entre los agentes y con el entorno, para poder observar comportamientos más complejos del sistema que forman los agentes y su entorno.

- Se parte de los elementos más pequeños y algo detallados para lograr formar sistemas más complejos.

- C. ¿Cuándo se usa el concepto de autoorganización?

- Cuando un sistema logra o obtiene orden mediante la interacción de sus componentes (pueden ser mediante la interacción de los agentes entre ellos o con el entorno).

- En algunas simulaciones de modelos lo podemos ver cuando surgen patrones de manera espontánea.

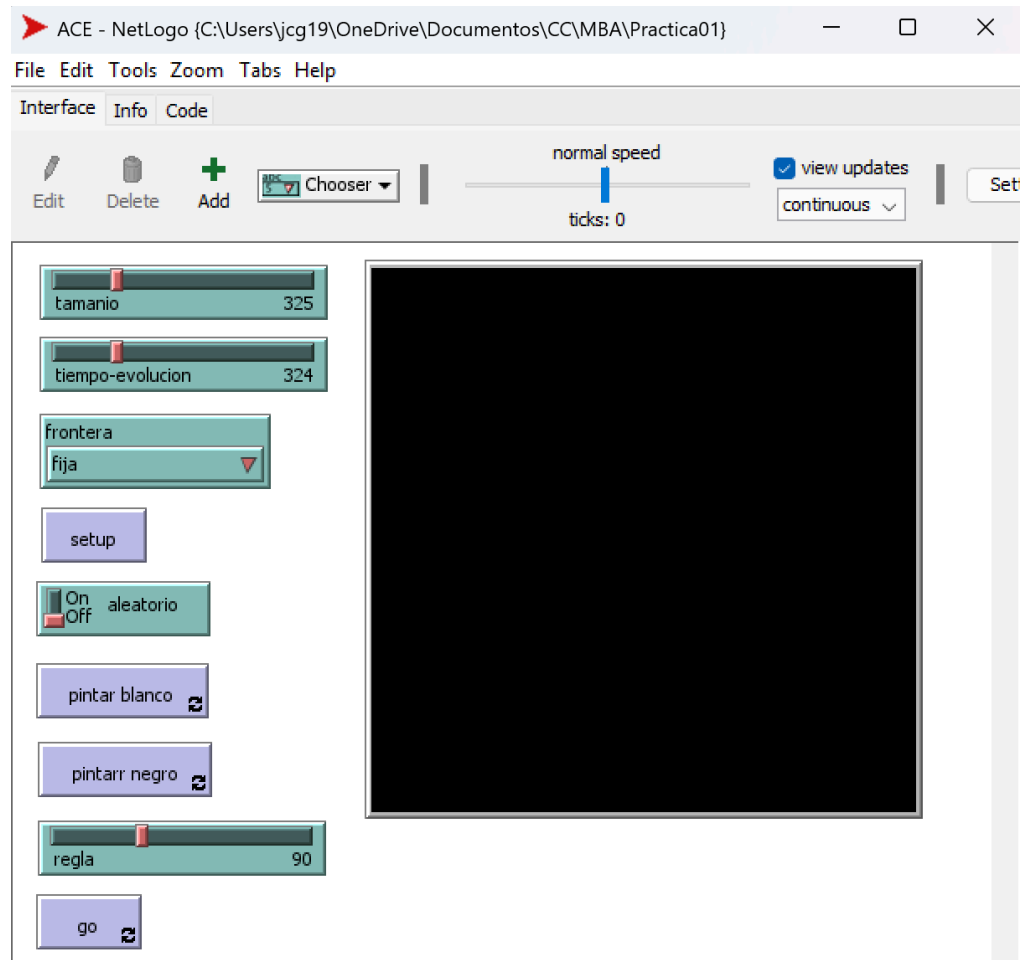
- D. ¿Qué es una propiedad emergente?

- Es una propiedad que tiene un sistema pero esta propiedad surge de la interacción entre los elementos del sistema, no es una propiedad que se encuentre en los componentes o partes del sistema.

- Parte 2

- ☐ Implementación de Autómata Elemental Celular

- Este se encuentra en el archivo llamado ACE



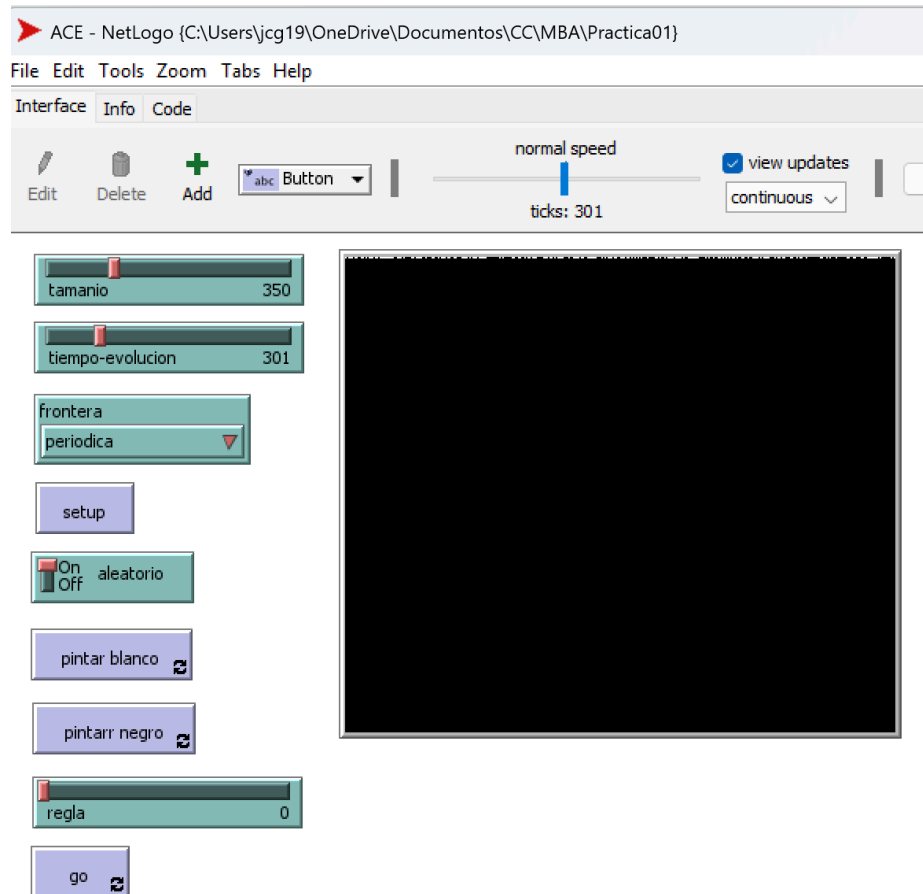
Primero se selecciona el tamaño y tiempo de evolución con los dos sliders de hasta arriba, después se puede seleccionar entre fronteras fijas o periódica con el chooser. Cuando se seleccionen estos tres parámetros se debe dar click en setup, lo cual creara el mundo con las condiciones y pintar el cuadro del centro de la fila de esta arriba de blanco (que un patch sea blanco significa que esta prendido), este cuadrado blanco unico en el centro es la condiccion inicial fija, si se quiere que la condiccion inicial sea aleatoria se puede seleccionar en el switch aleatorio (esto antes de dar setup) de esta manera obteniendo que la fila de hasta arriba tenga patches en blanco o negro de manera aleatoria. En caso de que se quiera una condición inicial personalizada (introducida por el usuario) se debe pintar esta, para esto luego de darle al botón de setup se puede dar al botón de pintar blanco y darle click a los patches que quieran que esten prendidos (en blanco), ademas se puedan usar el boton pintar negro para apagar patches.

Para elegir una regla se hace con el slider de hasta abajo, eligiendo un número, obsérvese que luego de elegir la regla se debe presionar el botón de setup para que la regla se cargue, y para que el autómata inicie se debe dar click en el botón de go.

Luego de cada ejecución (darle click al botón go) se debe volver a poner las condiciones iniciales con el botón setup y lo explicado arriba.

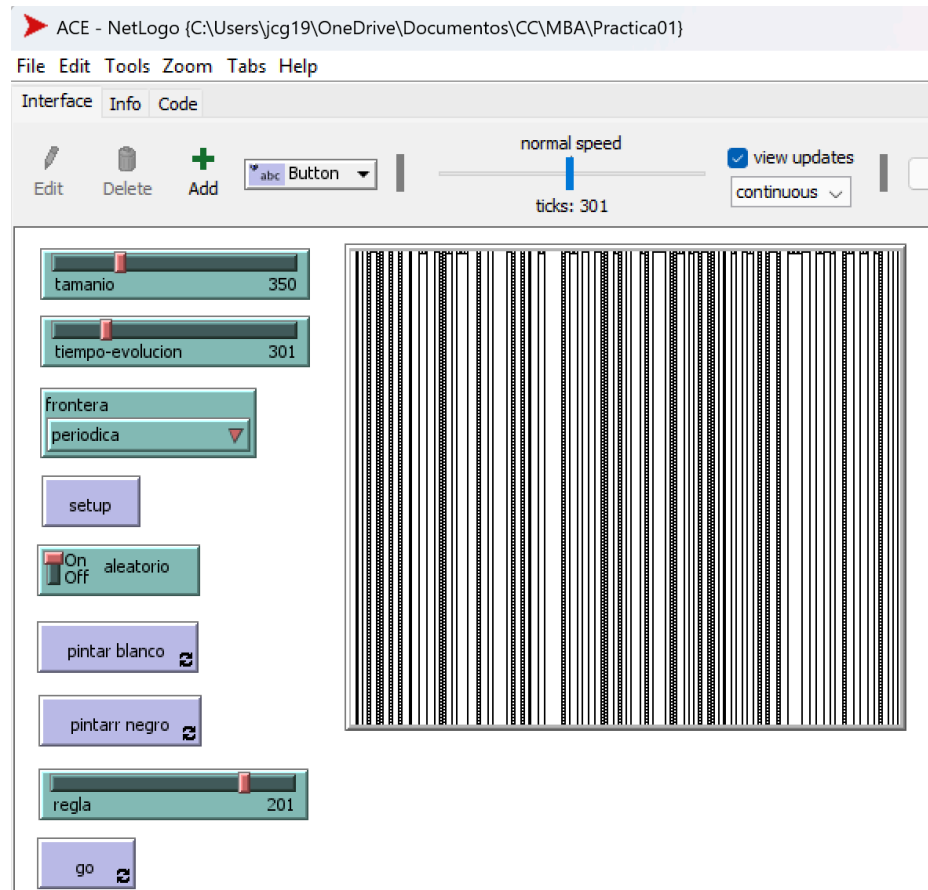
☐ Ejercicios

★ Exploración Clase 1



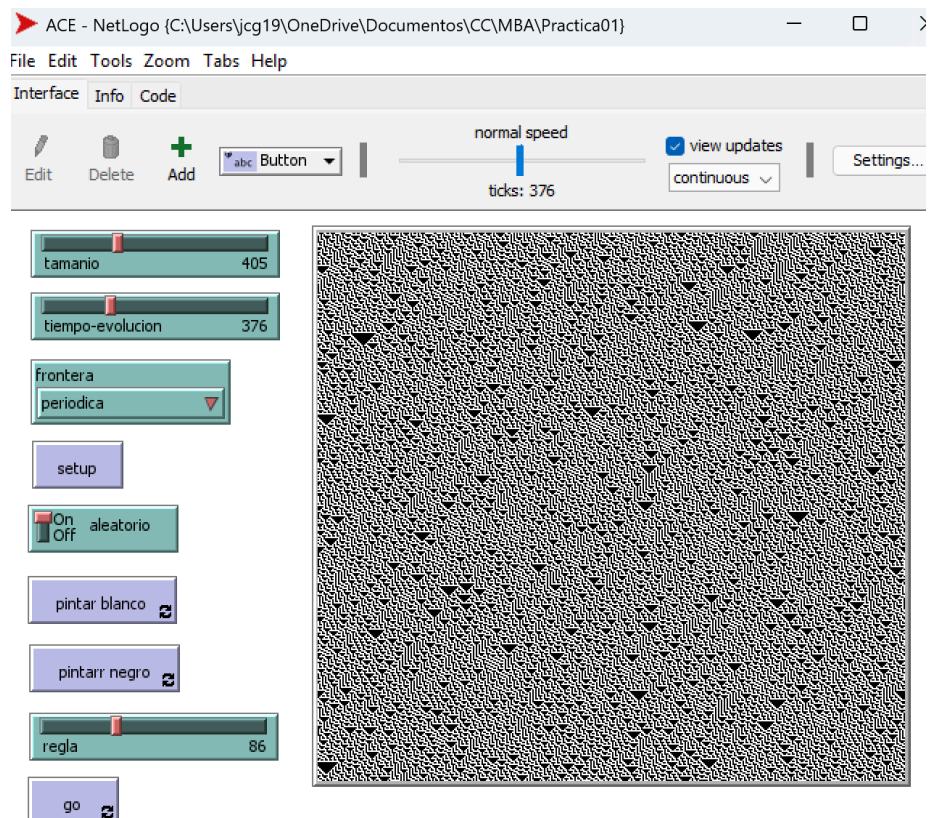
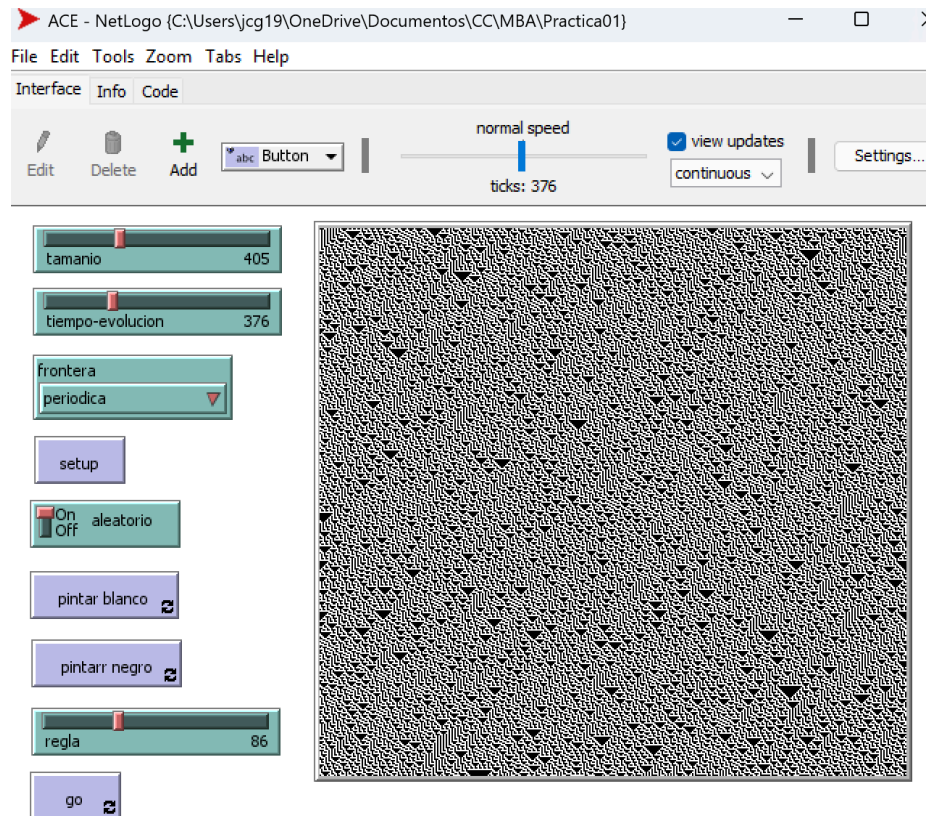
Aquí puse la regla 0, ya que luego de poca iteraciones todo el automata se pone en apagado (en negro) por lo tanto estamos llegando a algo homogéneo y estable, ya que llegamos a que todo este en negro y ya no cambiamos. Por lo tanto la dinamica de este automata es que todo se apaga (se pone en negro) y así se mantiene, ya no cambia.

Clase 2



Aquí puse la regla 201, ya que llegamos a estructuras/cosas periódicas o que se repiten, esto lo podemos ver en que se forman líneas verticales, y en algunas podemos ver que son como escaleras, es decir, que tiene un cuadratio blanco redondeado de cuadrados negros (en sus 8 vecinos), así los cuadraditos blancos redeados de los negros se siguen repitiendo a lo largo de la línea vertical. Por lo tanto la dinámica de este autómata es que se forman tanto líneas verticales todas negras y también líneas negras verticales las cuales tiene "hoyos" que son cuadraditos blancos, algo similar a escaleras o una cinta de cine.

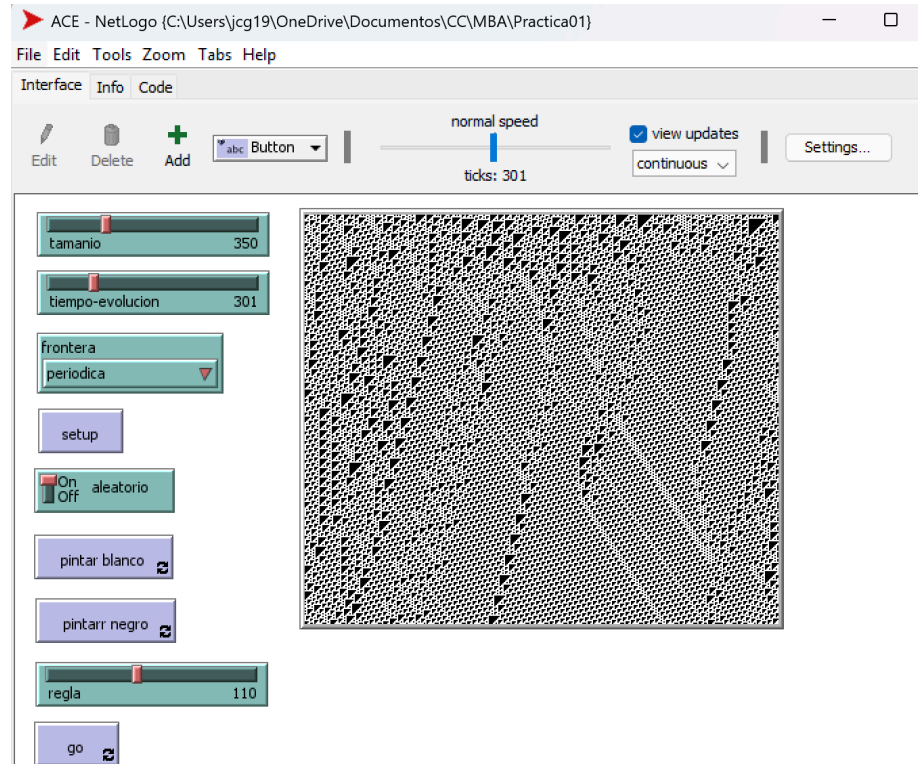
Clase 3



Aquí puse la regla 86, ya que luego de varias iteraciones podemos ver que no se forman patrones y la aleatoriedad se mantiene a lo largo de las iteraciones, esto lo podemos notar en que en ciertos puntos se forman triángulos negros de diferentes tamaños. Por lo tanto la dinámica de este autómata es que se forman diferentes triángulos de

cabeza los cuales varían de tamaño, además podemos notar que se forman como figuras con forma de “L” las cuales también varían de tamaño y posición.

Clase 4



Aquí puse la regla 110, ya que se van formando estructuras que mediante las iteraciones avanzan estas estructuras se propagan, esto lo podemos notar en cómo algunos triángulos se van desplazando a lo largo de las iteraciones. Por lo tanto la dinámica de este autómata es que se forman triángulos ladeados los cuales parece que a lo largo de tiempo se van desplazando hacia la izquierda o hacia abajo, otra cosa interesante es que algunas veces cuando estos triángulos que se van desplazando chocan con otros objetos estos se destruyen y en algunos casos forman como líneas. Además en la clase del día 20/08/25 vimos que esta regla era un ejemplo de la clase 4, pero lo más importante es que es equivalente a una máquina de Turing universal.

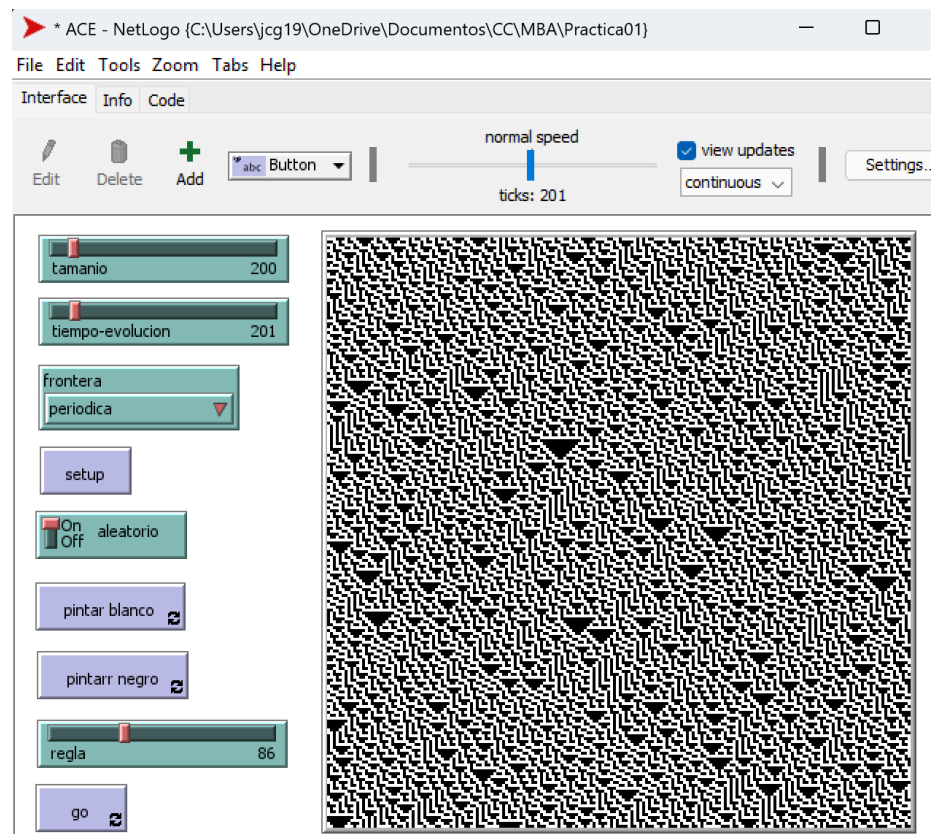
Y por último sabemos que de la clase 1 hay 24 reglas, de la clase 2 hay 194 reglas, de la clase 3 hay 26 reglas y de la clase 4 hay 12. Esto lo sabemos gracias a la clase del día 20/08/25.

★ Sensibilidad a las condiciones iniciales

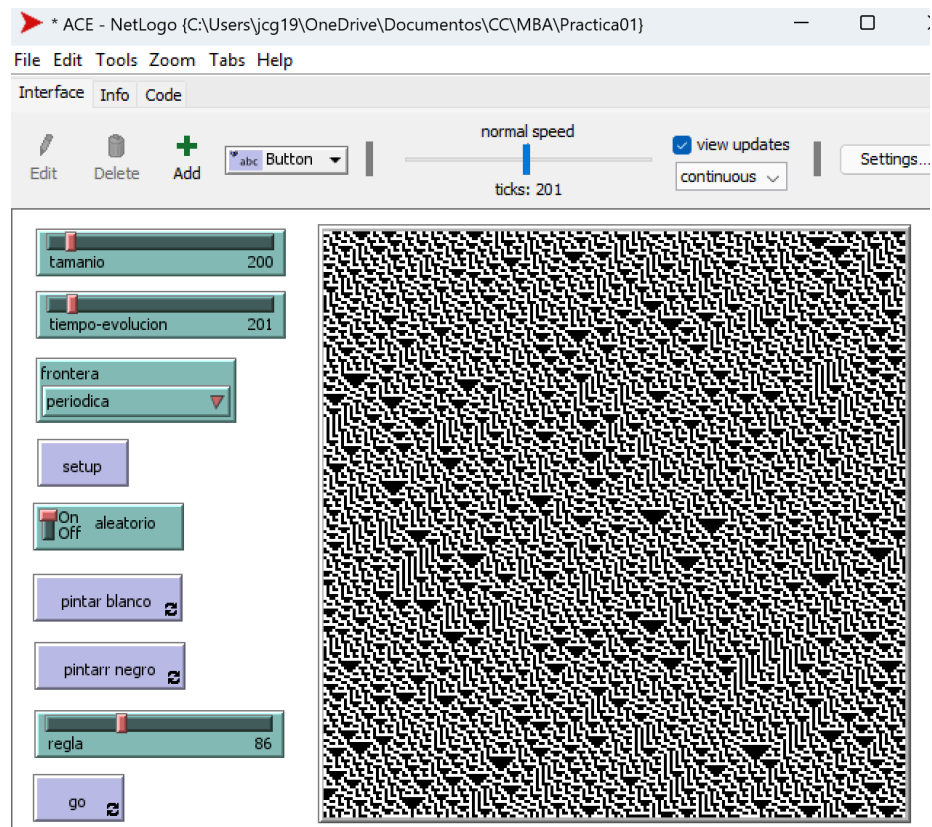
Para esto use la regla 86, primero ejecute el autómata con una configuración inicial aleatoria (configuracion1.csv), el resultado es la primera imagen y se encuentra en sin-perturbar.csv, luego a la configuración inicial le cambie un bit (configuracion2.csv) y ejecute el

automa, el resultado es la segunda imagen y se encuentra en perturbado.csv. Después use el programa diferencias.py para poder ver las diferencias de los csv y en las coordenadas donde los colores no eran iguales los puse de color verde y lo guarde en diferencias.csv, por último cargue las diferencias y como resultado salió la tercera imagen, donde está pintado de color verde las diferencias de los estados, además lo que está de color negro y blanco es lo que si está igual en ambos estados.

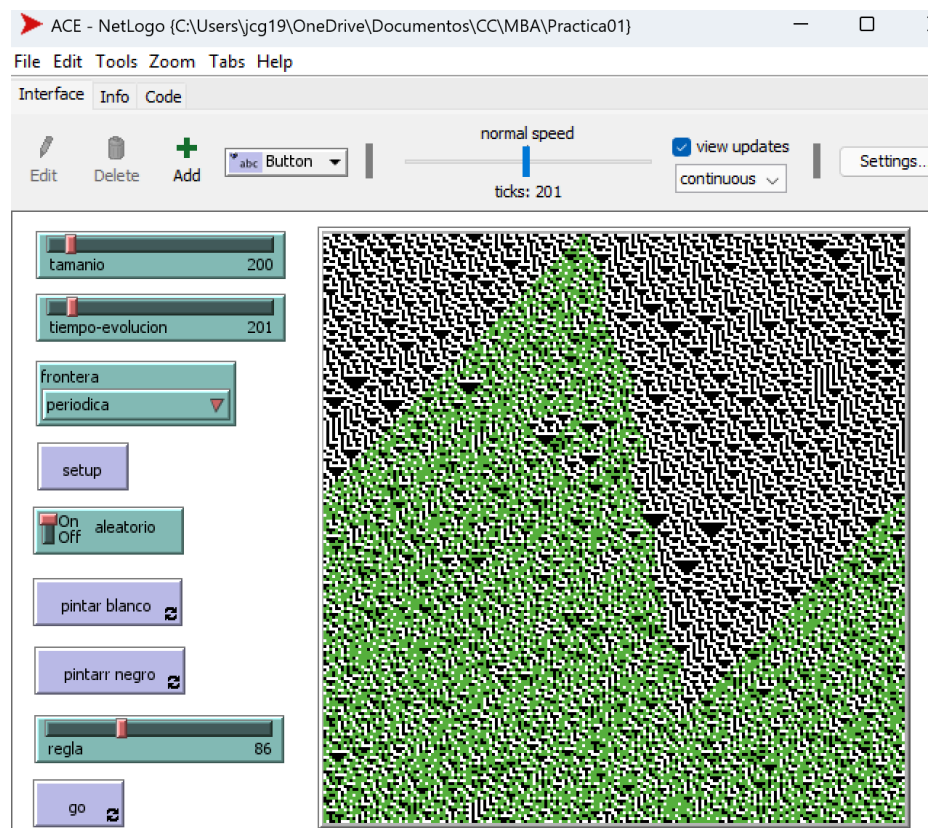
Sin perturbar



Perturbado



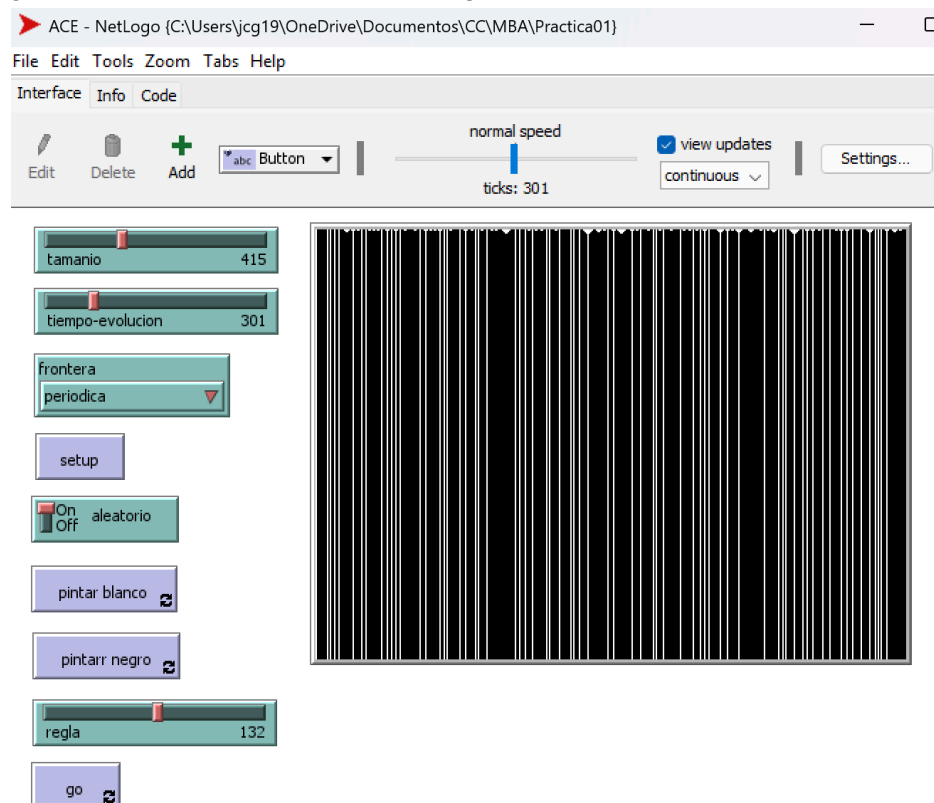
Diferencias

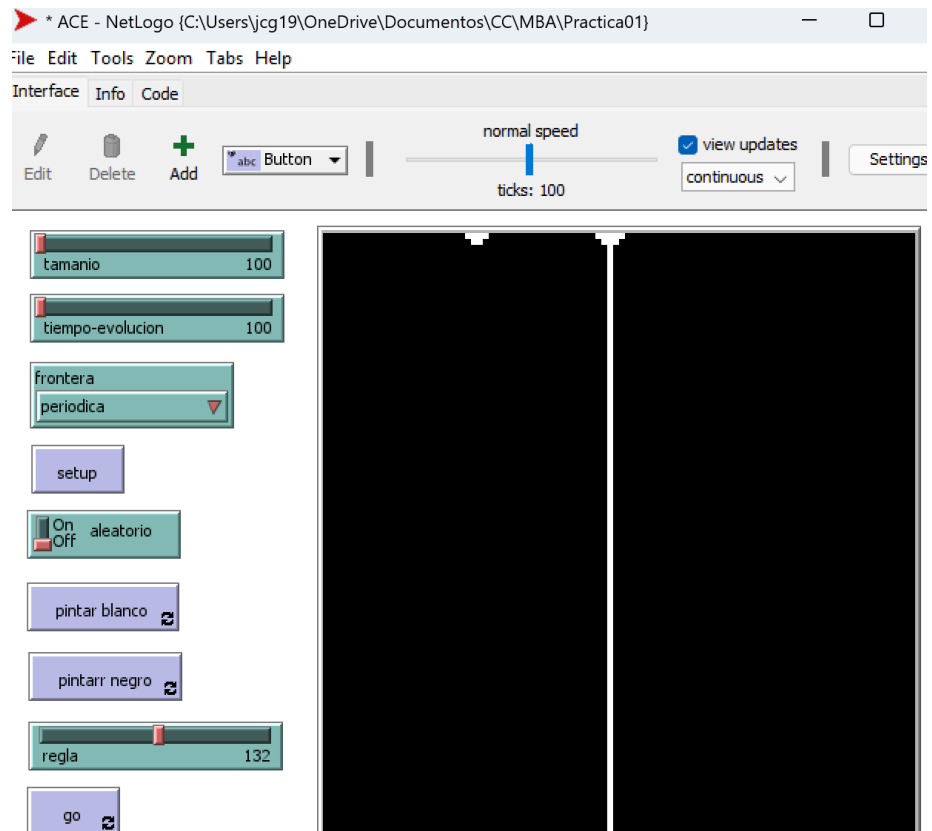


Como podemos ver esta regla si es sensible a las condiciones iniciales ya que un pequeño cambio (un bit) fue creando muchas diferencias.

¿La sensibilidad a las condiciones iniciales es una propiedad suficiente para catalogar la dinámica como caótica? No, es una condición necesaria para decir que la dinámica es caótica, esto debido a que podemos observar cómo pequeños cambios (en este caso un cambio de un bit) provoca grandes cambios a los resultados, se está cumpliendo que cambios pequeños generan grandes efectos (como es mencionado en la lectura 1). Al principio pensaba que si era suficiente pero estuve investigando algo y encontré que se necesitan de otras dos propiedades las cuales son transitividad topológica (es decir que el sistema se puede mover o desplazar en el espacio si se le da suficiente tiempo) y puntos periódicos densos (una cantidad suficiente o grande de puntos a los cuales el sistema regresa luego de cierto tiempo o iteraciones), esto según la definición un sistema dinámico caótico propuesta por Robert L. Devaney.

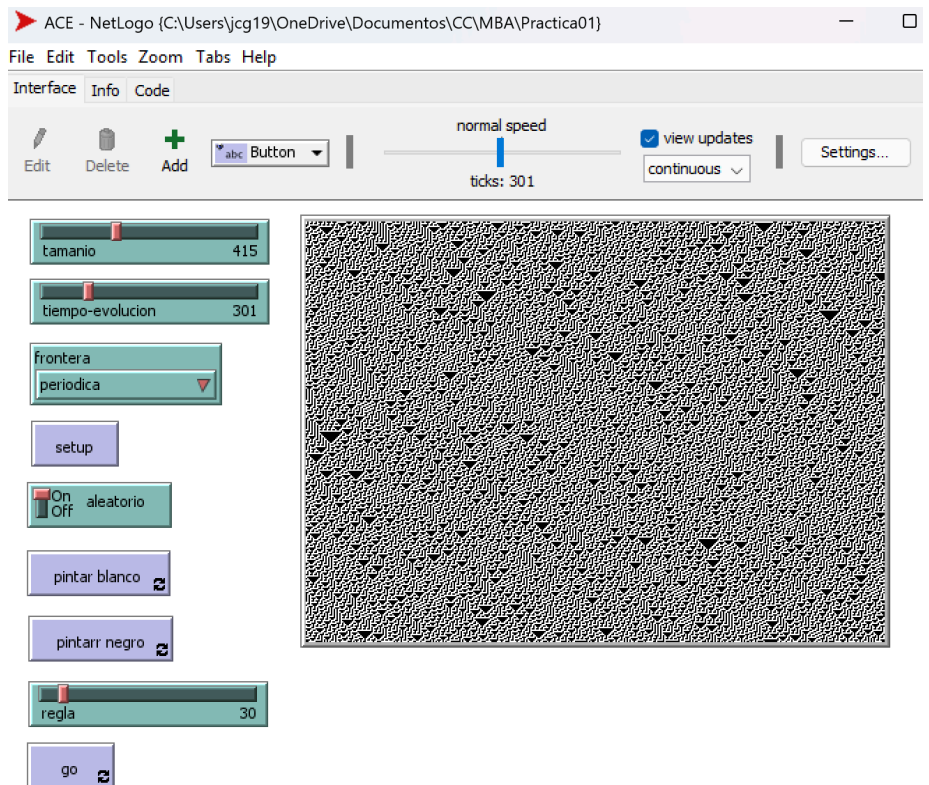
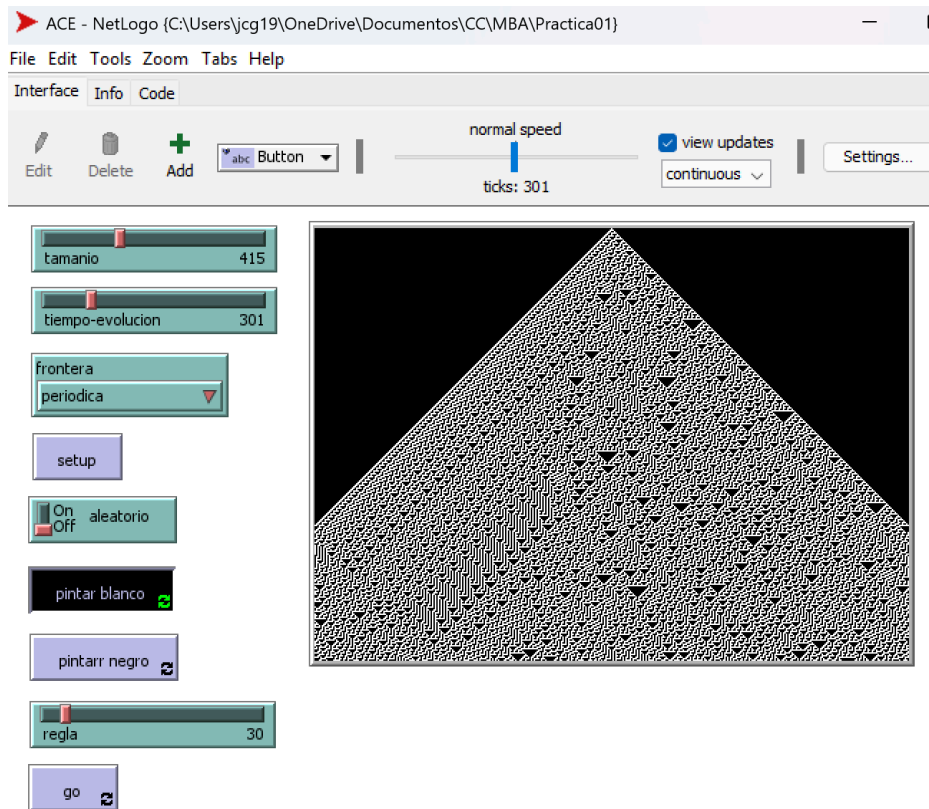
★ ¿Qué poder de cómputo tiene la regla 132?

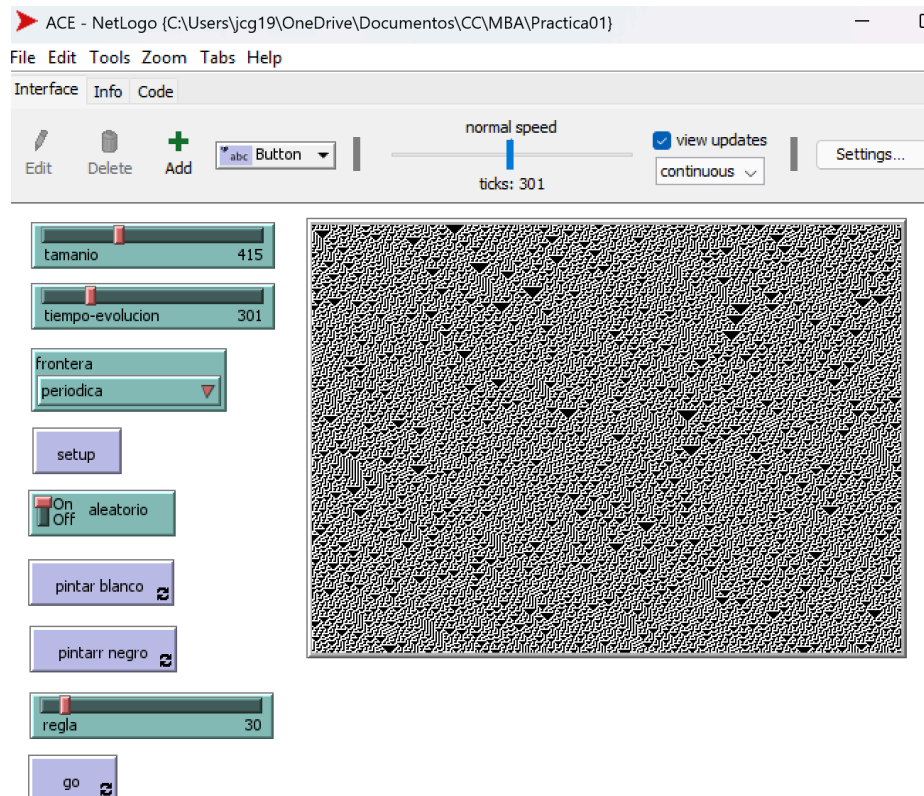




Lo que puedo notar de esta regla es que es de clase 2, ya que luego de varias ejecuciones podemos ver que se forman líneas verticales las cuales se mantienen y se repiten. Sobre esta regla otra cosa que note es que la condición inicial no afecta cuantas líneas verticales largas se van formar, ya que si tenemos un conjunto de cuadrillos blancos juntos donde la cantidad de cuadrillos blancos juntos es impar (por ejemplo tenemos tres celdas o cuadrillos blancos juntos y a los extremos cuadrillos negros, ■□□■) entonces se formará una línea vertical blanca que nace de la mitad y además se van reduciendo este conjunto de cuadrillos blancos formando una pirámide invertida hasta llegar a realizar la línea vertical larga. Algo similar pasa con los conjuntos de cuadrillos blancos de tamaño par, en este caso solo se forma la pirámide invertida pero no la línea vertical larga. Por lo tanto este autómata nos puede servir para saber la paridad de los conjuntos de cuadrillos blancos juntos.

¿Qué propiedades tiene la regla 30?





Primero podemos notar que la regla 30 es de clase 3 esto debido a que podemos ver que durante las iteraciones la aleatoriedad se mantiene (imagen 2), no podemos encontrar patrones. También podemos notar que dependiendo de la configuración inicial puede cambiar mucho lo que resulta de las iteraciones. Otra propiedad interesante es que esta regla es usada para generar números aleatorios enteros largos en Wolfram Language, esto debido a que es caótica, como pudimos notar ya que lo que se muestra es afectado por la condición inicial. Entonces algo interesante es que esta regla la podemos usar para generar cosas “aleatorias” ya que al darle diferentes condiciones iniciales obtenemos resultados diferentes los cuales podemos interpretar para obtener valores. Otra cosa es que es algo similar a la regla 86, ya que se siguen formando triángulos bocabajo de diferentes tamaños y en diferentes tiempos, solo que aquí se forman figuras con forma de “L” pero volteada a la izquierda.

★ Imágenes

Para empezar por lo que podemos ver en la segunda figura podemos pensar que la regla 22 es de clase 3, esto debido a que la aleatoriedad de la condición inicial se ve reflejada y se mantiene durante las iteraciones o evolución. Notemos que en la primera imagen podemos ver que todo parte del centro (la celda encendida), parecer tener una simetría, y podríamos pensar que tiene una frecuencia algo larga (un triángulo grande hacia arriba relleno y un triángulo grande hacia abajo vacío, así repitiendo). Ahora observando la segunda imagen podemos ver que no logramos encontrar ningún patrón o frecuencia, toda la aleatoriedad de la condición inicial se

mantiene y va generando diferentes triángulos de varios tamaños (esto lo podemos notar algo fácil viendo los triángulos vacíos que tienen diferentes tamaños y ubicaciones).

Entonces podemos notar que este autómata es sensible a la configuración inicial, si esta cambia entonces el resultado también y pueden pasar cambios muy grandes, el autómata de la regla 22 es caótico.

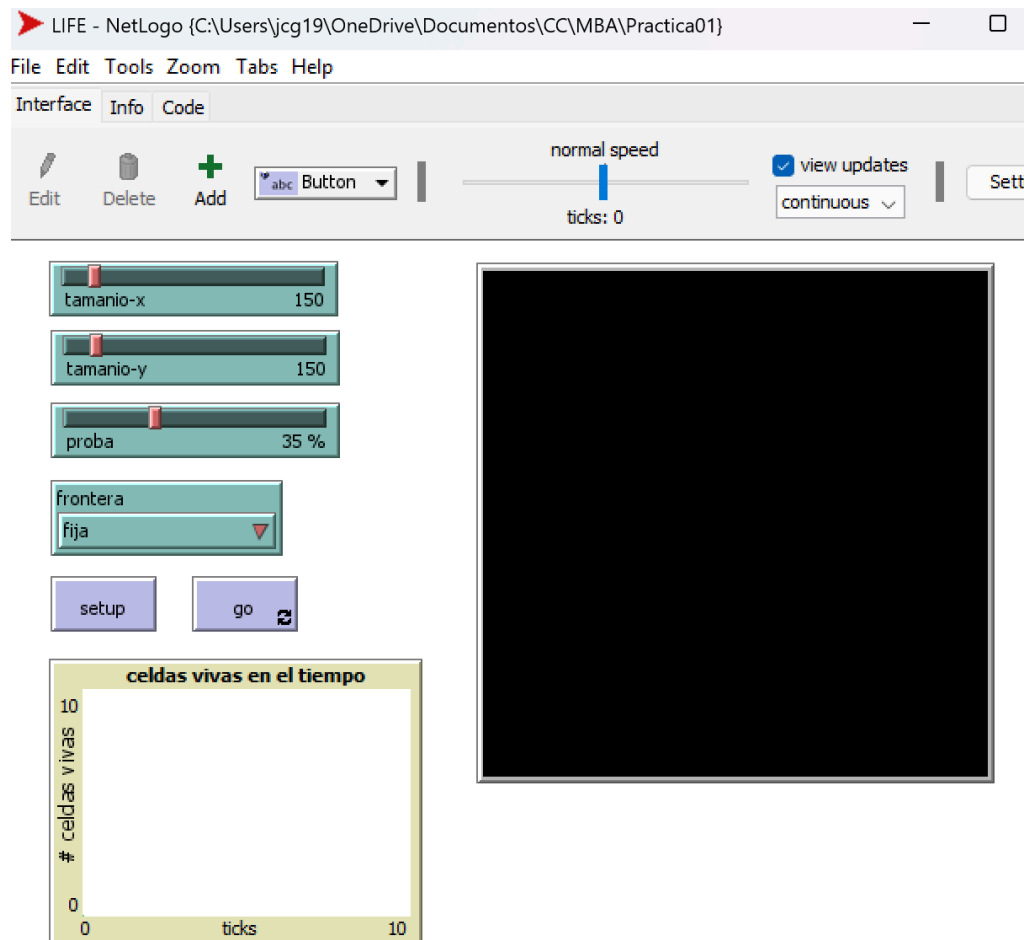
Notemos que la dinamica en la primera imagen es formar triángulos partiendo de la condicion inicial donde solo hay un cuadrado prendido (en negro) y luego se van formando más triángulos dentro del triángulo más grande, ahora observemos que en la segunda imagen debería pasar algo similar ya que son la misma regla, pero lo que pasa es que como las condiciones iniciales son aleatoria entonces se empiezan a formar muchos triángulos más pequeños a diferencia de la primera imagen, no vemos que en la segunda imagen los triángulos parecen que estan sobrepuestos unos de otros, esto se debe a las condiciones iniciales. Aunque en ambas imágenes podemos notar triángulos. Algo más que nos ayuda a entender esta regla es que solo vamos a tener que una cuadrado seguirá prendido o vivo si en el tiempo anterior se tiene que entre sus vecinos de los lados y el cuadrado solo uno está prendido o vivo, de esta manera en la primera imagen como solo hay un cuadrado prendido al inicio entonces la construcción del triángulo grande puede iniciar sin ningún conflicto pero cuando la condición es aleatoria como en la segunda imagen tenemos que el triángulo grande no se va a formar, sólo se forman algunos pequeños, ya que hay varios puntos de donde empiezan a formarse triángulos.

- ★ Si el autómata estuviera definido en un alfabeto de 3 estados considerando sus primeros vecinos, ¿cuántas posibles reglas hay?
Primero notemos que si usamos los primeros vecinos y un alfabeto de 3 estados entonces tenemos que existen $3^3 = 27$ posibles configuraciones para la celda y sus primeros vecinos, ya que en total son 3 celdas y cada celda puede tomar uno de tres valores de estado, ahora como cada posible configuración puede dar como resultado uno de los tres estados tenemos que hay 3^{3^3} posibilidades, entonces tenemos que hay $3^{3^3} = 3^{27}$ posibles reglas.

- Parte 3

- ☐ Implementación de LIFE

- Este se encuentra en el archivo llamado LIFE



Primero se selecciona el tamaño para eje x y uno para el eje y con los dos sliders de hasta arriba, luego se selecciona el porcentaje de condición inicial aleatorio (que porcentaje de que una celda esté viva al inicio) con el slider que dice proba, después se puede seleccionar entre fronteras fijas o periódica con el chooser. Cuando se seleccionen estos parámetros se debe dar click en setup, lo cual creara el mundo con las condiciones, notemos que cuando una celda esta de color verde significa que esta viva y cuando no esta viva entonces la celda esta de color negro.

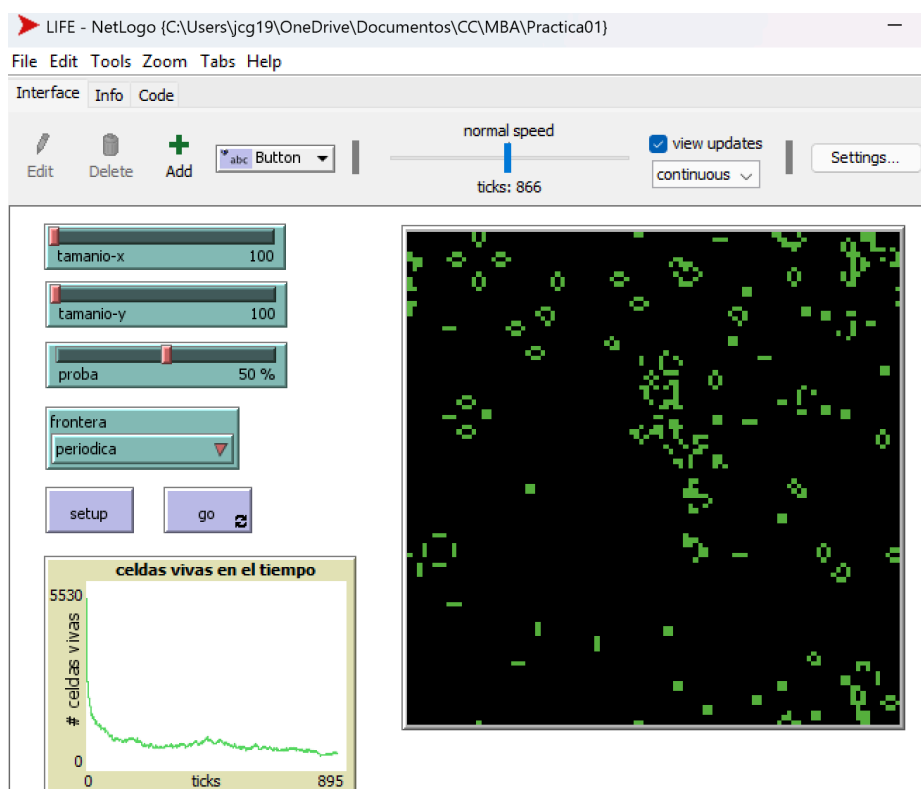
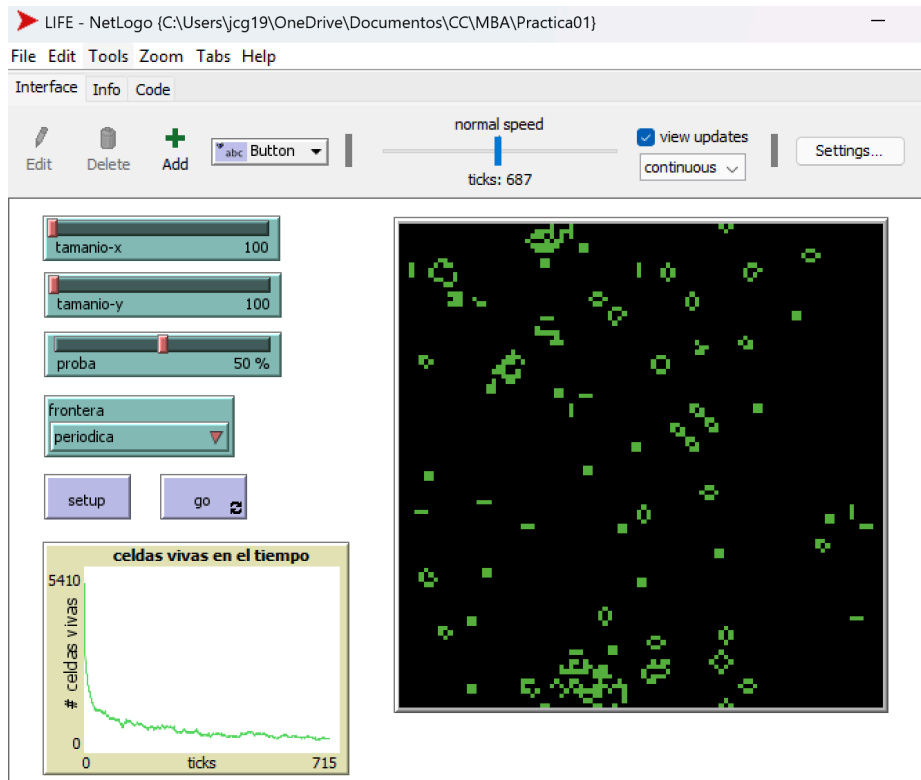
Para empezar la simulación de debe dar click se debe dar click en el botón de go.

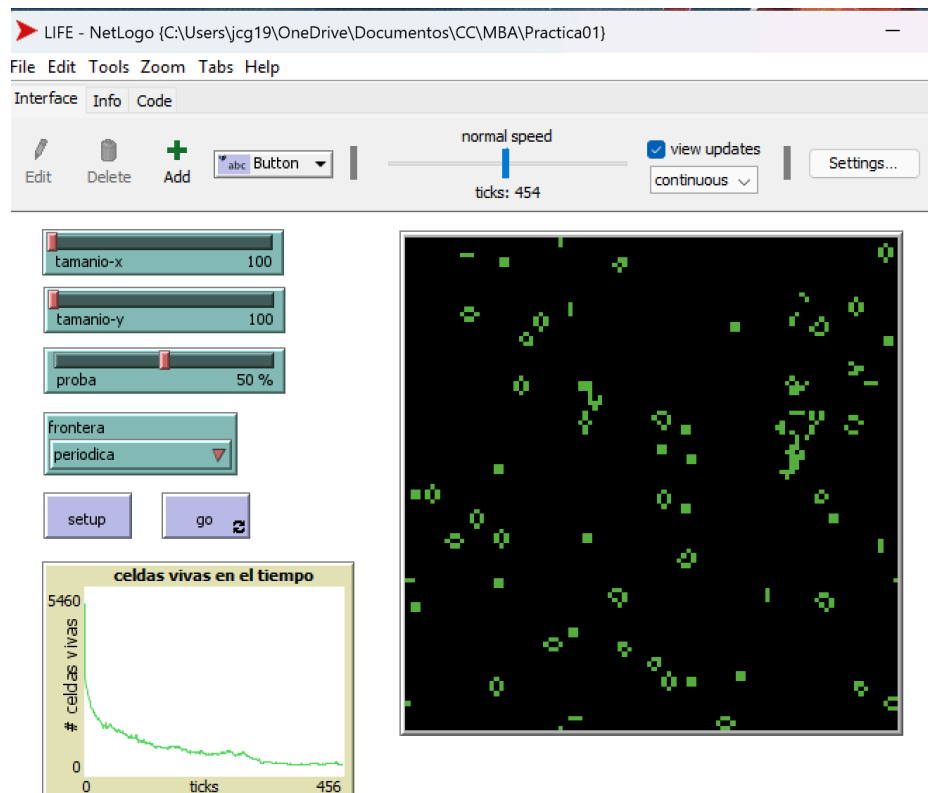
Abajo del botón de go se encuentra una gráfica donde se lleva el conteo de la cantidad de celdas vivas a través de los ticks.

□ Ejercicios

★ Ejecuciones

Con 50% por lo general se estabiliza (aquí estabiliza lo estoy tomando cuando en la gráfica ya no hay cambios tan grandes aunque no sea completamente una línea) a los 425 ticks (masones en algunos casos se estabiliza alrededor los de 350 ticks y en otros alrededor de los 550 ticks). Las siguientes son 3 imágenes de ejecuciones.

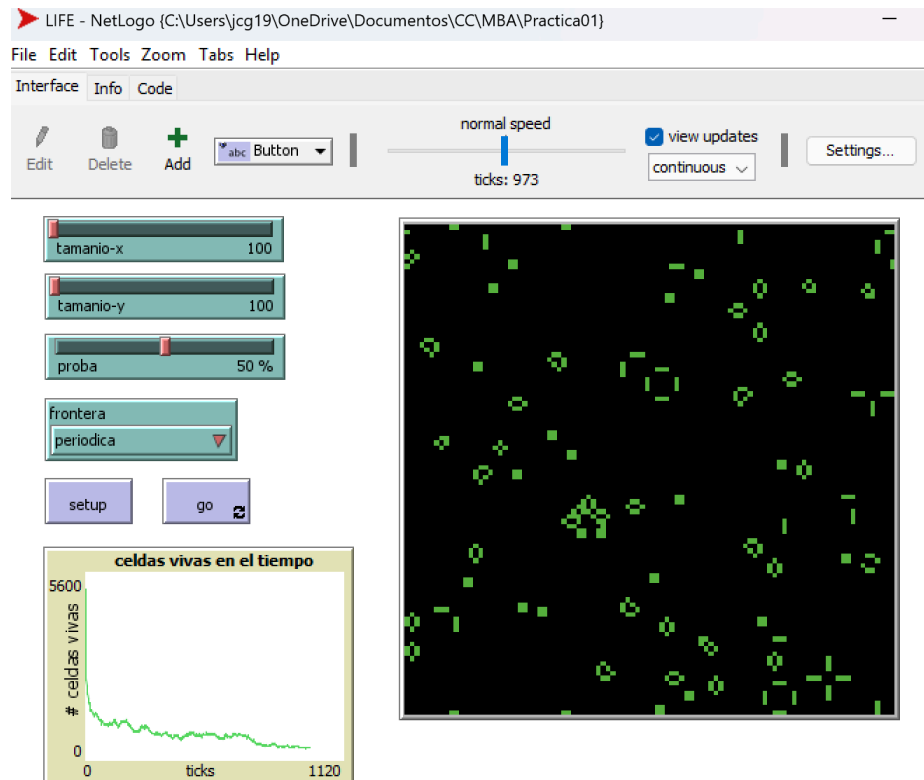




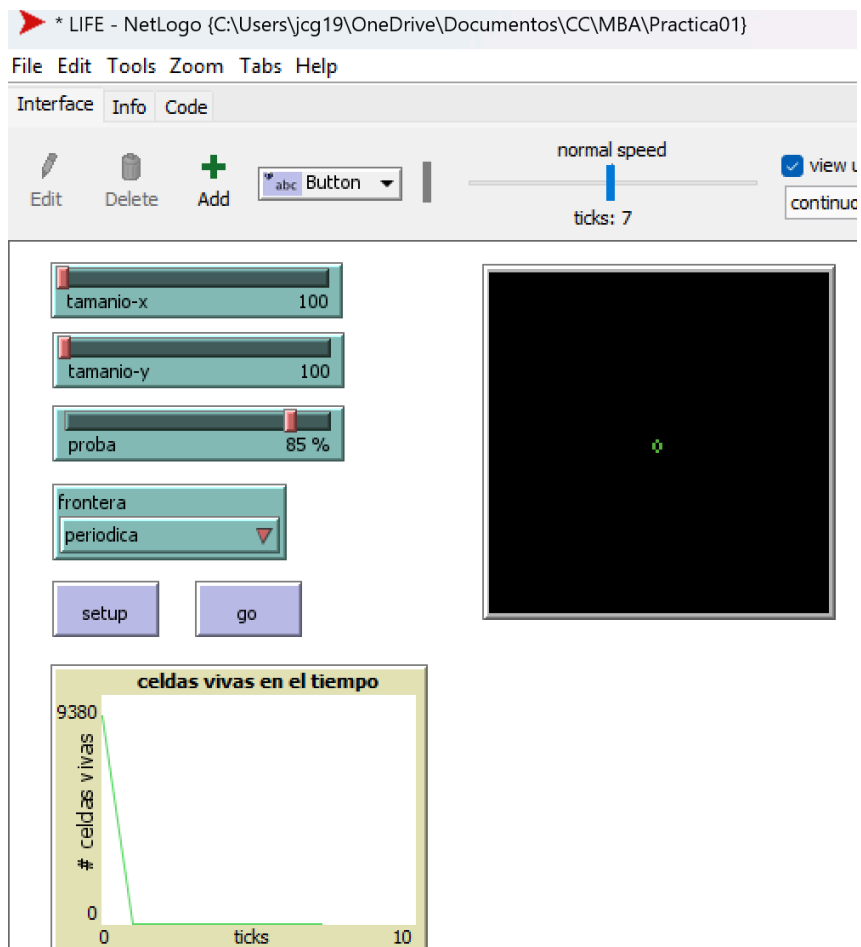
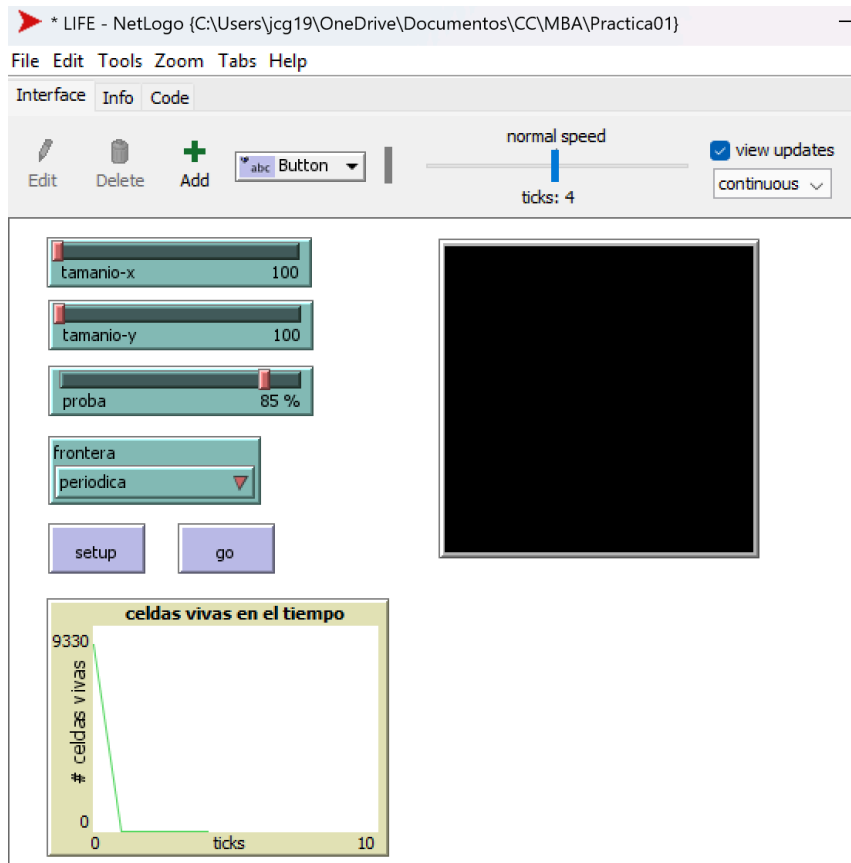
¿Para cualquier configuración inicial aleatoria pasará lo mismo?

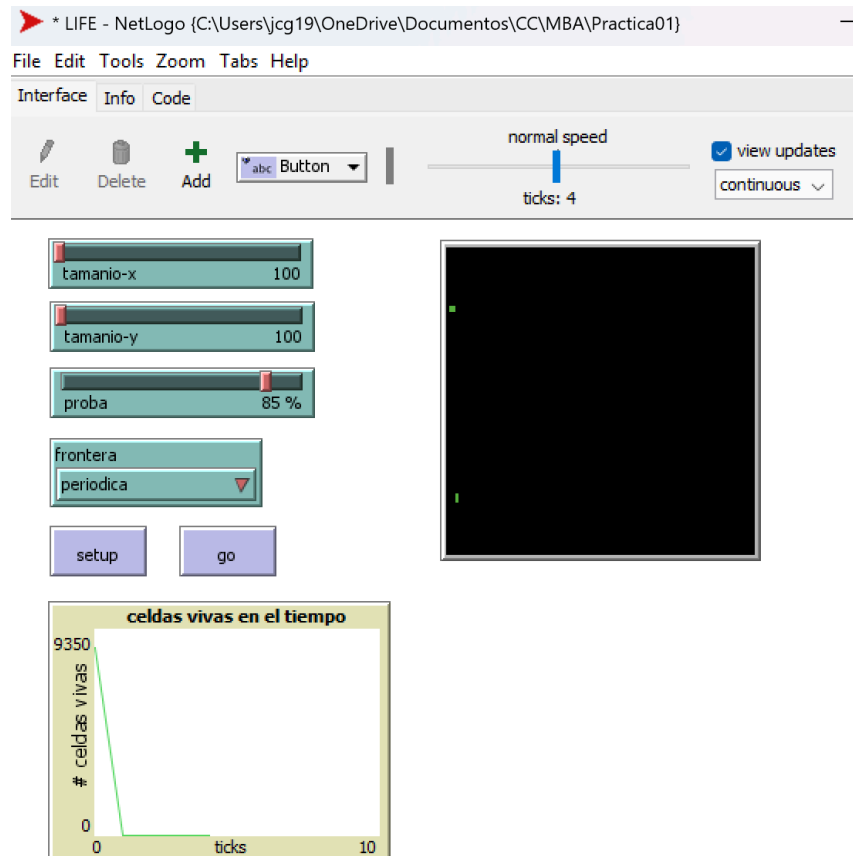
Para toda configuración inicial no pasa siempre la misma cantidad de ticks en algunas puede llegar a ser más o mucho menos, hubo un caso donde antes de los 300 ticks ya se había estabilizado pero otro donde vi que como por los 700 ticks era donde se empezaba a estabilizar, por lo tanto no siempre va a pasar lo mismo con las configuraciones iniciales aleatorias, tomarán diferente cantidad de tiempo para estabilizarse pero al menos en todas las ejecuciones que realice si se llegaron a estabilizar.

Lo que sí pude notar es que la mayoría del tiempo se van formando figuras muy similares en todas las ejecuciones.



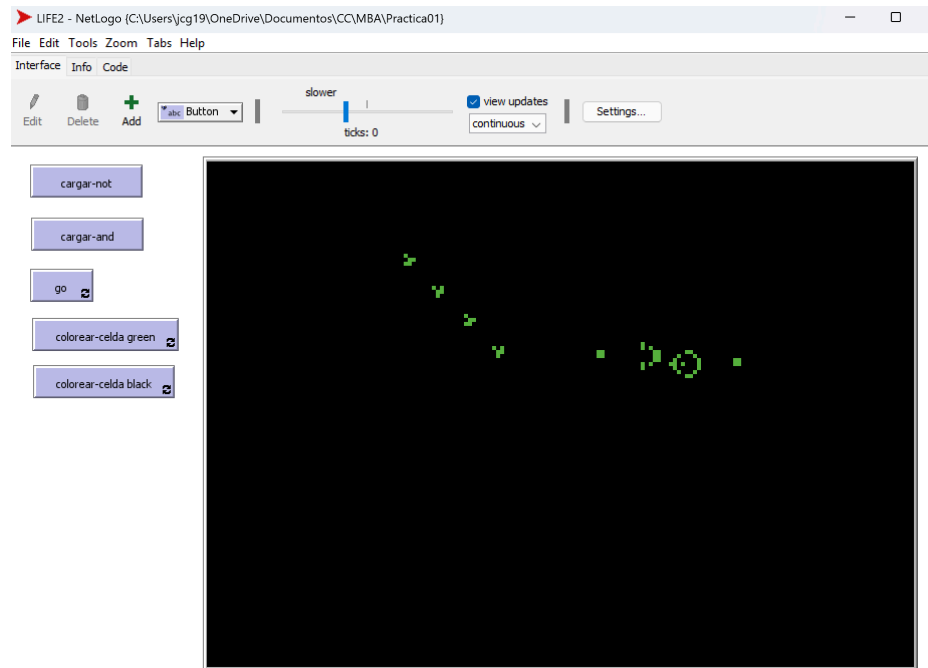
Con 85% por lo general se estabiliza a los 1 o 2 ticks (en algunos casos podían llegar a los 3 ticks para estabilizarse), en general aquí se estabiliza demasiado rápido. Generalmente en el primer tick todas las celdas se mueren, pero en algunos casos cuando sobreviven algunas entonces en el segundo tick las demás siguen muriendo, hasta llegar que al tick 3 o 4 todas las celdas están muertas (en algunos casos esto no pasa, como una imagen de abajo). Las siguientes son 3 imágenes de ejecuciones.





¿Para cualquier configuración inicial aleatoria pasará lo mismo?
Aquí no importaba tanto la configuración inicial aleatoria ya que a los pocos ticks muchas celdas morían (en el primer tick casi todas las celdas mueren) y solo una cantidad muy reducida seguía con vida. Esto creo que pasa ya que al inicio hay mucha cantidad de celdas vivas por lo tanto en general la gran mayoría de celdas moría durante los dos primeros ticks. Entonces la dinámica con 85% fue muy diferente a la de 50%, ya que con 85% se llega a estabilizarse muy rápido pero la cantidad de celdas vivas que quedan es muy pequeña (en algunos casos no sobrevive nadie), en cambio con 50% se tarda más en estabilizarse pero al final hay más celdas vivas.

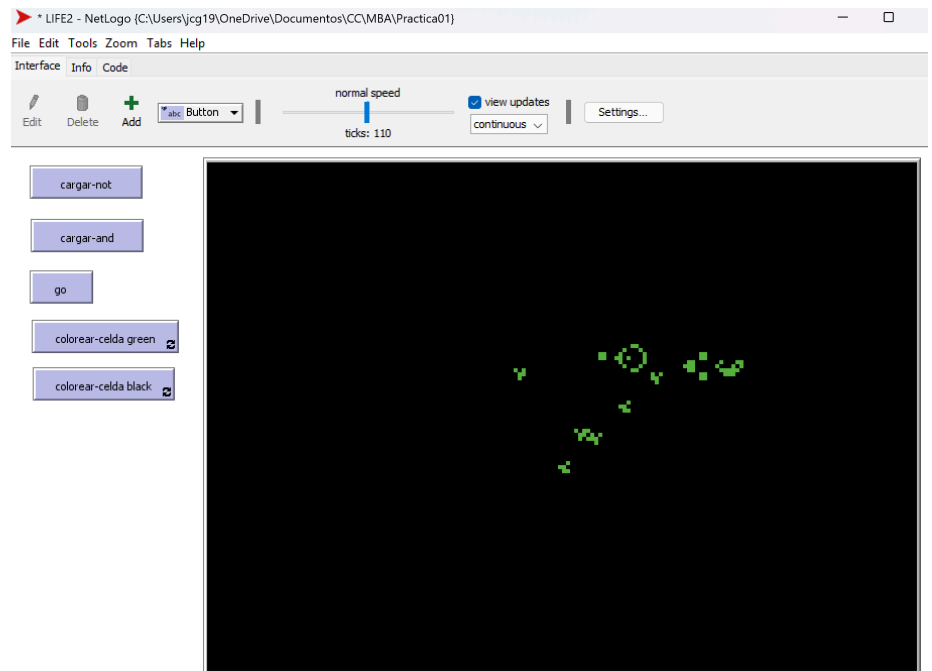
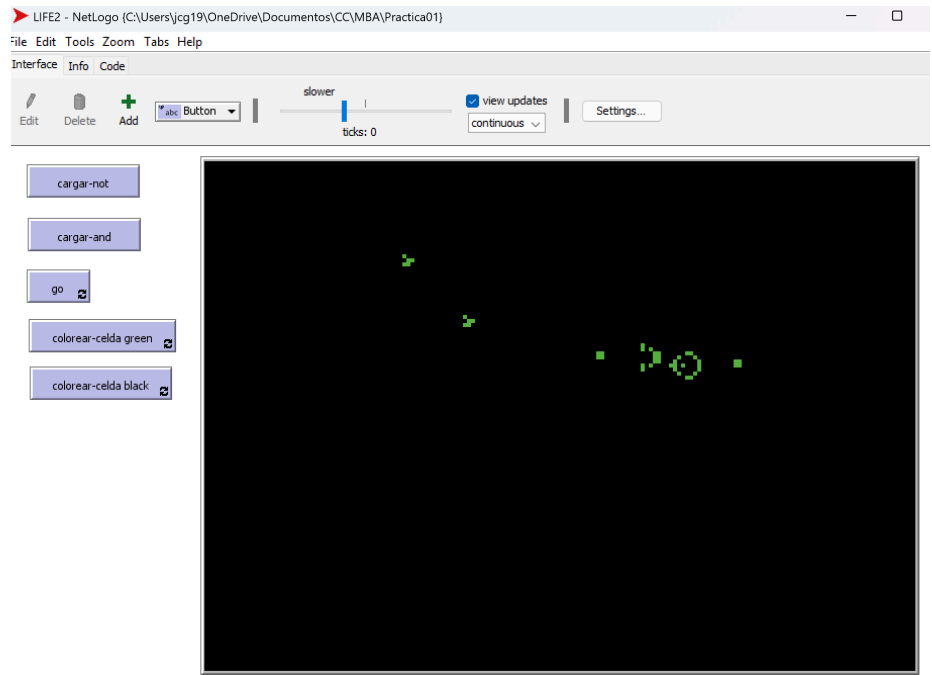
- ★ Compuerta NOT
Este se encuentra en el archivo llamado LIFE2

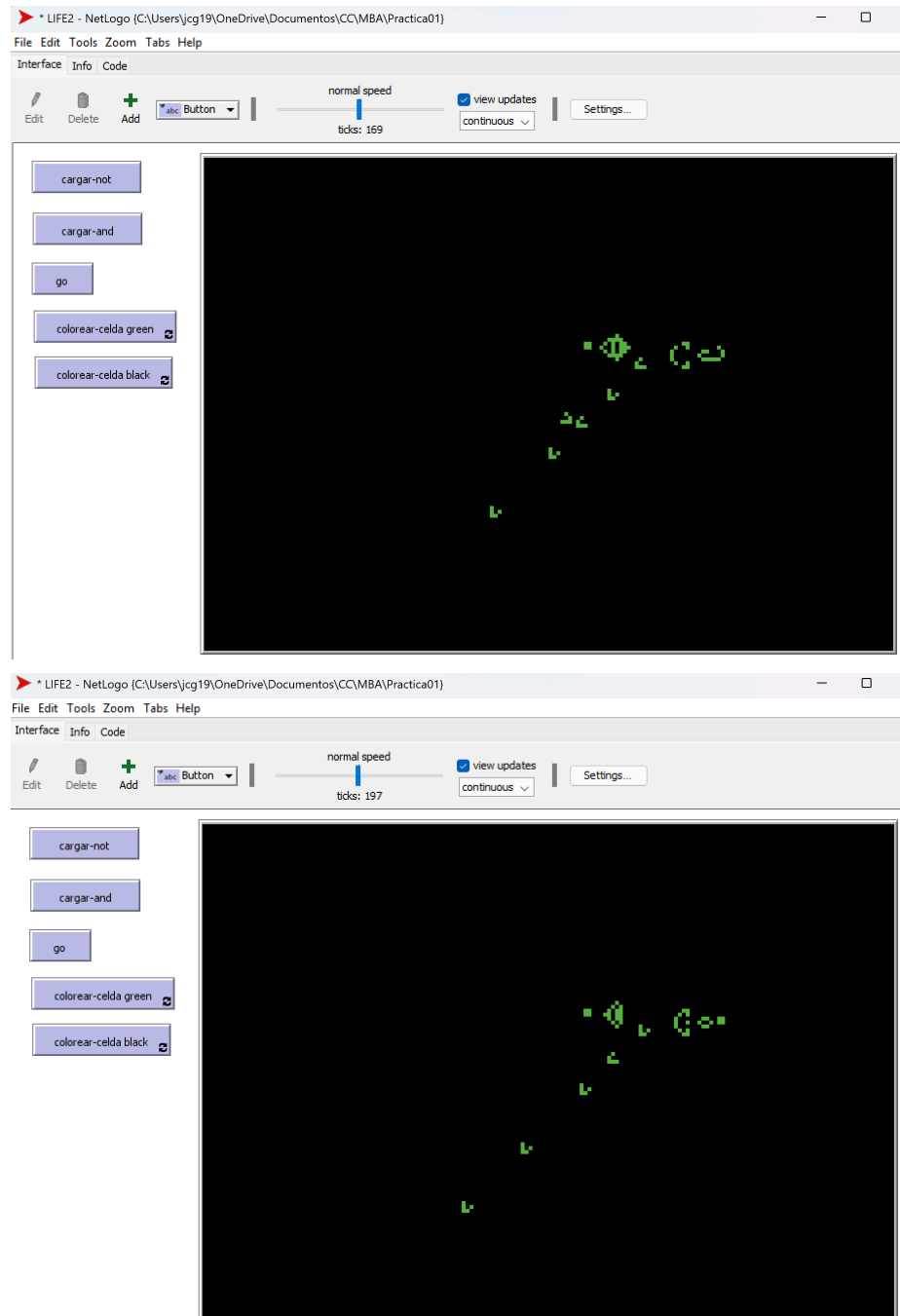


Para ver la implementación de la compuerta NOT se debe dar click en el botón cargar-not, cuando se de click el mundo se creara con la compuerta, notemos que cuando una celda esta de color verde significa que esta viva y cuando no esta viva entonces la celda esta de color negro. En la parte izquierda del mundo se encuentran 4 gliders los cuales representan a A (usando la imagen del diagrama) por lo tanto existen A_1 (el glider más cercano a la pistola), A_2 , A_3 y A_4 (el glider más arriba), cada glider en verde representa que es true, por lo tanto si se quiere que un A_i sea false lo que se tiene que hacer es “borrarlo” usando la función colorear-celda black. El resultado de la puerta NOT serán los gliders que sobrevivan de la pistola (de igual manera los gliders que viven, en verde, son true y si no hay glider entonces es false).

Para empezar la simulación se debe dar click se debe dar click en el botón de go.

Un ejemplo de la simulación es este, donde empezamos con $A_1 = false$, $A_2 = true$, $A_3 = false$ y $A_4 = true$, por lo tanto obtenemos una salida de *true*, *false*, *true* y *false* (esto lo podemos observar viendo que el primer glider de la pistola sigue vivo debido a que no chocó con A_1 , el segundo ya no existe debido a que chocó con A_2 , el tercero sigue vivo debido a que no chocó con A_3 y el cuatro ya no existe debido a que chocó con A_4).





¿Qué implicaciones tiene controlar ciertas estructuras en función del tiempo?

Una de las implicaciones que note al momento de implementar la compuerta es que como están en un función del tiempo necesitamos que todos los componentes de las compuertas estén en el tiempo adecuado, es decir que todos estén sincronizados, por ejemplo que los gliders generados choquen en el momento adecuado para que puedan destruirse, por lo tanto sincronizar todos los componentes de las compuertas puede ser complicado, y luego juntar más compuertas puede ser algo nada trivial. Otra implicación es que entre cada “señal”

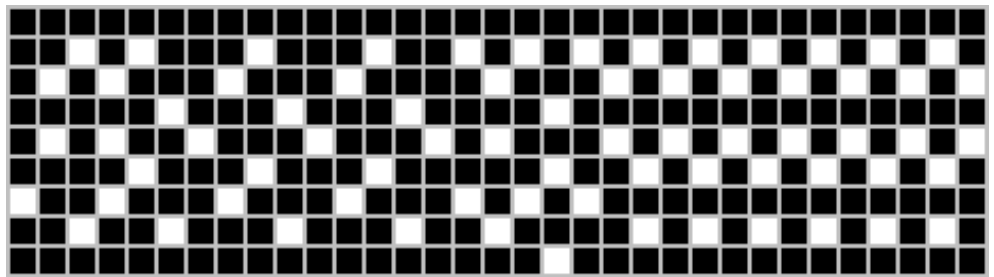
hay un tiempo, el cual usa los gliders para recorrer su camino, por lo tanto la respuesta de estas compuertas es algo lenta.

Otra cosa es que si al momento de la implementación o sincronización existen errores con el tiempo esto puede causar resultados no deseados o hasta que todo el sistema quede sin funcionar (se destruya).

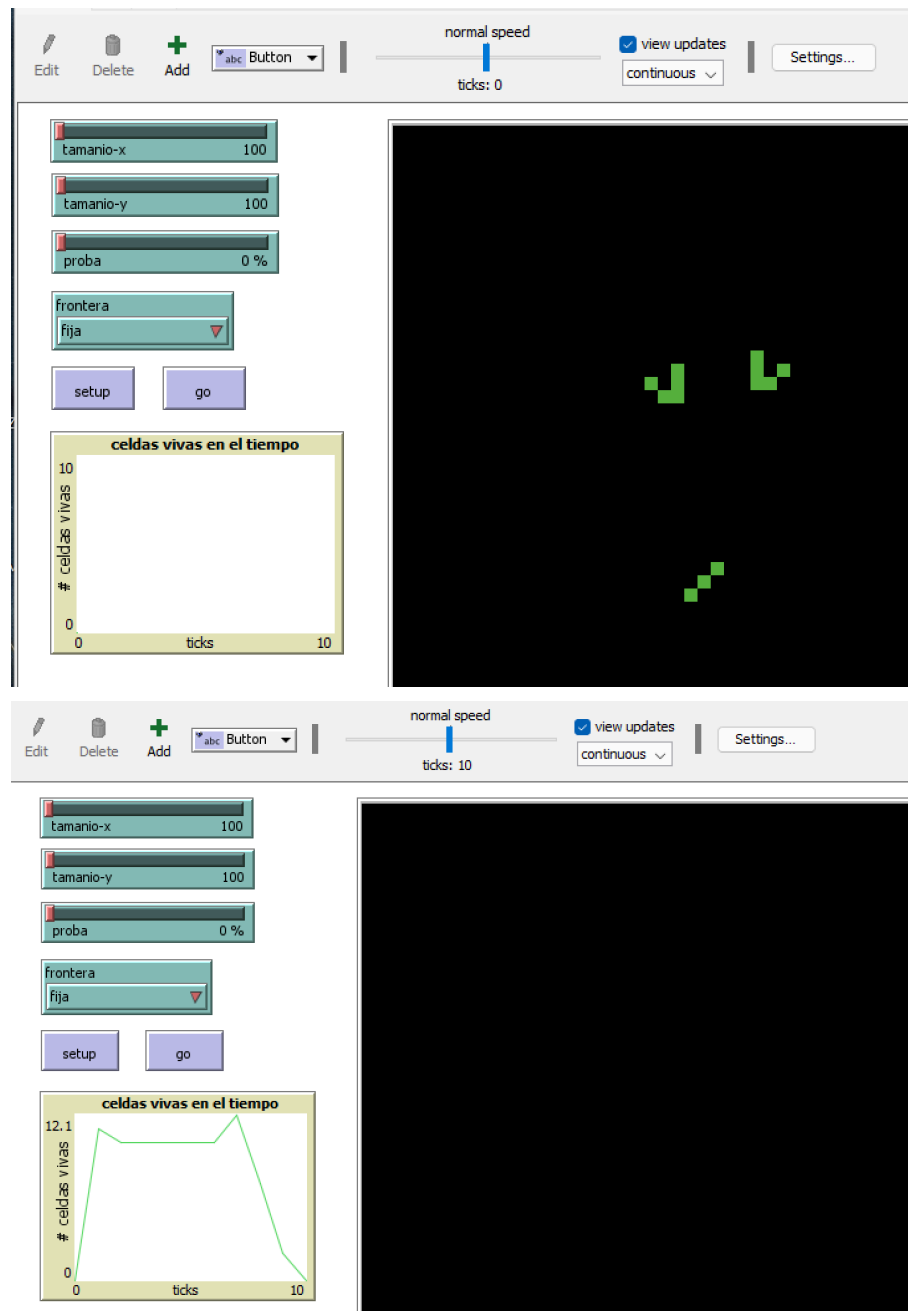
★ ¿El juego de la vida es reversible?

No es reversible, ya que si fuera reversible podríamos encontrar una serie de reglas que funcionaria como la inversa de las reglas que conocemos, como si le sacaramos la inversa a una función.

Notamos que para que el juego de la vida fuera reversible necesitamos que para cada estado existiera una configuración inicial de la cual podemos llegar a ese estado, pero esto no pasa. En la clase del 25/08/25 se presentó el Jardín del Edén, que es una configuración o estado del juego de la vida (imagen de abajo) para el cual no existe una configuración inicial (sin contar al mismo) del cual podemos llegar al Jardín del Edén, por lo tanto no todos los estados tienen una configuración inicial (sin contar al mismo estado) de la cual podemos partir para llegar al estado.

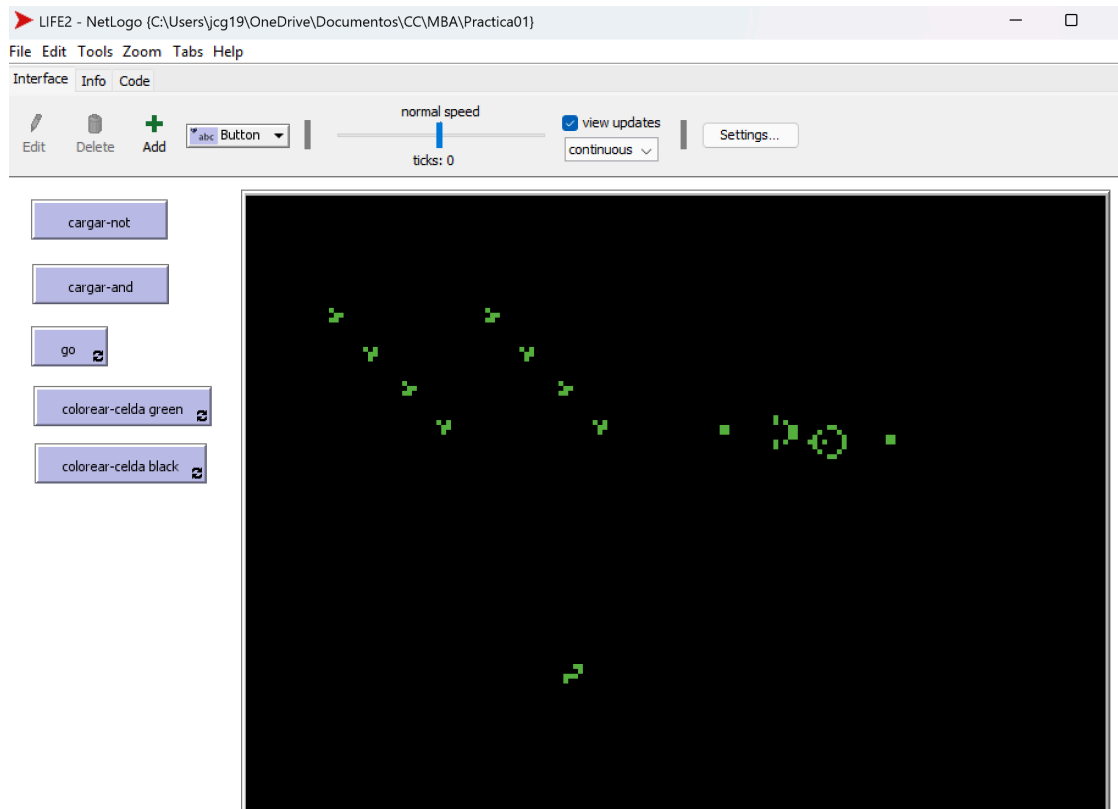


También necesitamos que desde dos estados no llegamos al mismo estados después de cierta cantidad de tiempo, esto para que desde un estado podamos generar una única configuración o estado, al similar como las funciones uno a uno, esto para poder hacer posible la reversión desde un estado, pero esto no siempre pasa, por ejemplo en la imagen de abajo tenemos dos gliders como la primera configuración y tres celdas vivas como la segunda configuración, luego de evolucionar ambas configuraciones podemos ver que llegamos al mismo estado, donde todas las celdas están muertas (esto se puede ver en el archivo blinkers-chocan.csv usando la función cargar-archivo, que vimos en las ayudantías), por lo tanto el estado donde todas las celdas están muertas lo podemos alcanzar desde dos configuraciones iniciales distintas.

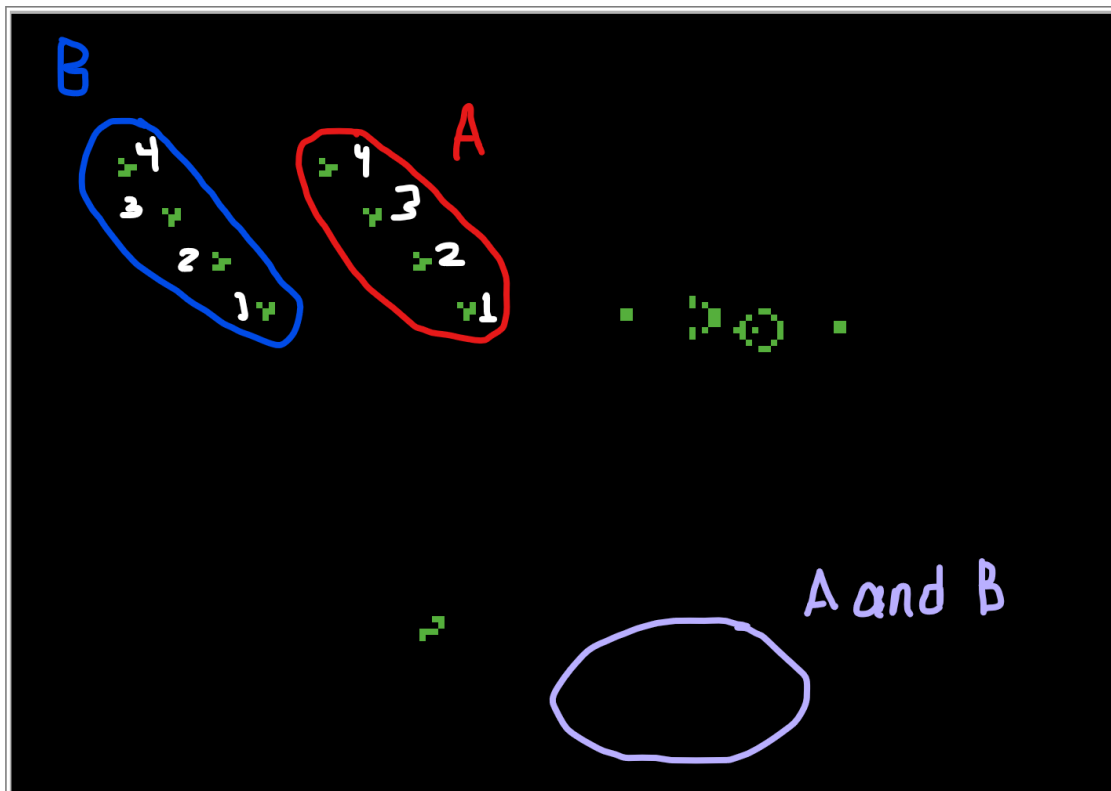


Por estas razones podemos concluir que el juego de la vida no es reversible.

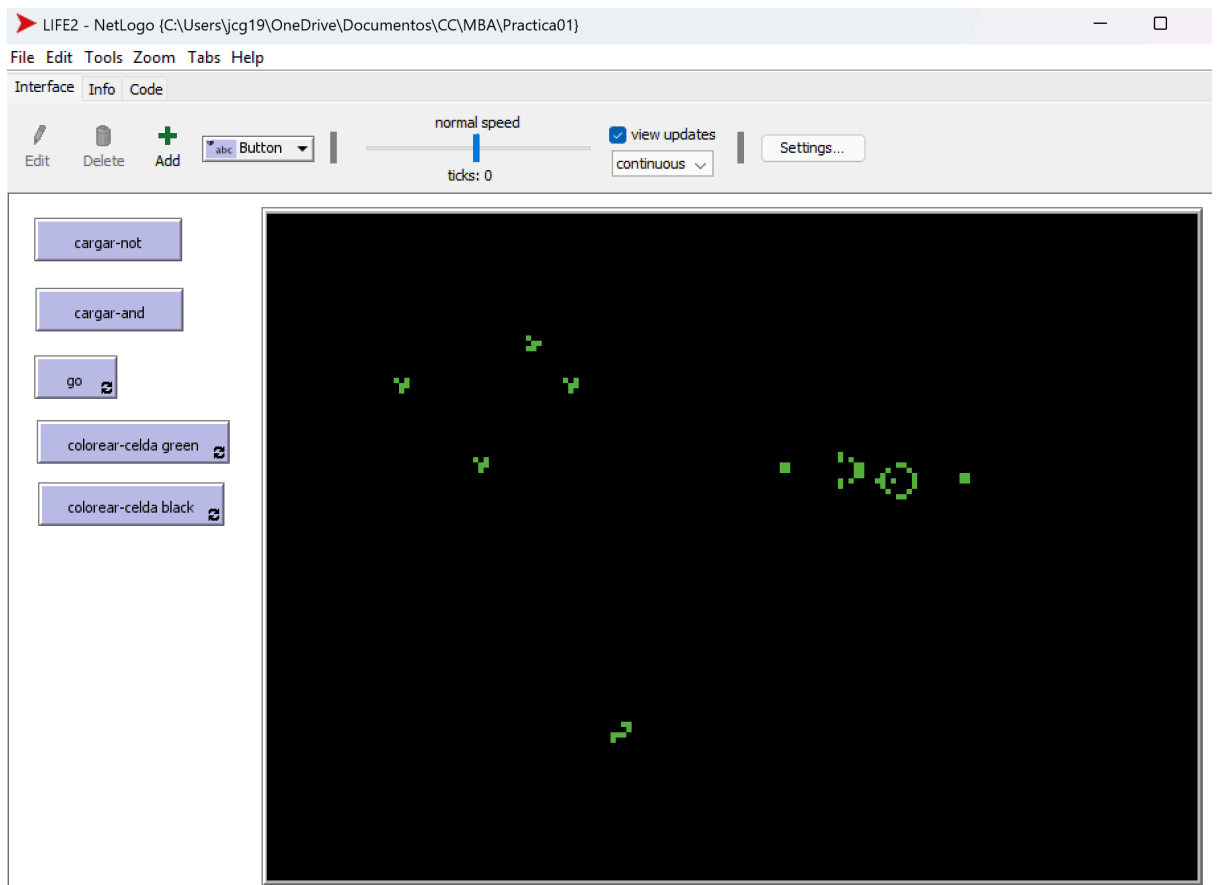
- Parte 4
Compuerta AND
Este se encuentra en el archivo llamado LIFE2



Para ver la implementación de la compuerta AND se debe dar click en el botón cargar-and, cuando se de click el mundo se creara con la compuerta, notemos que cuando una celda esta de color verde significa que esta viva y cuando no esta viva entonces la celda esta de color negro. En la parte más izquierda del mundo se encuentran 4 gliders los cuales representan a B (usando la imagen del diagrama) por lo tanto existen B_1 , B_2 , B_3 y B_4 (el glider más a la izquierda, se pueden ver los números en la imagen de abajo), cada glider en verde representa que es true, por lo tanto si se quiere que un B_i sea falso lo que se tiene que hacer es "borrarlo" usando la función colorear-celda black, de manera similar en el centro hay 4 gliders los cuales representan a A por lo tanto existen A_1 , A_2 , A_3 y A_4 , que los gliders en verde representan true se sigue manteniendo y de igual manera los podemos borrar para que representen false. El resultado de la puerta AND serán los gliders B que sobrevivan de la pistola, es decir los gliders que llegue a la zona marcada de morado en la imagen, notemos que si un glider hubiera llegado en un tiempo pero no llega entonces significa que el resultado de la compuerta es false, en cambio si llega un glider verde entonces el resultado es true.

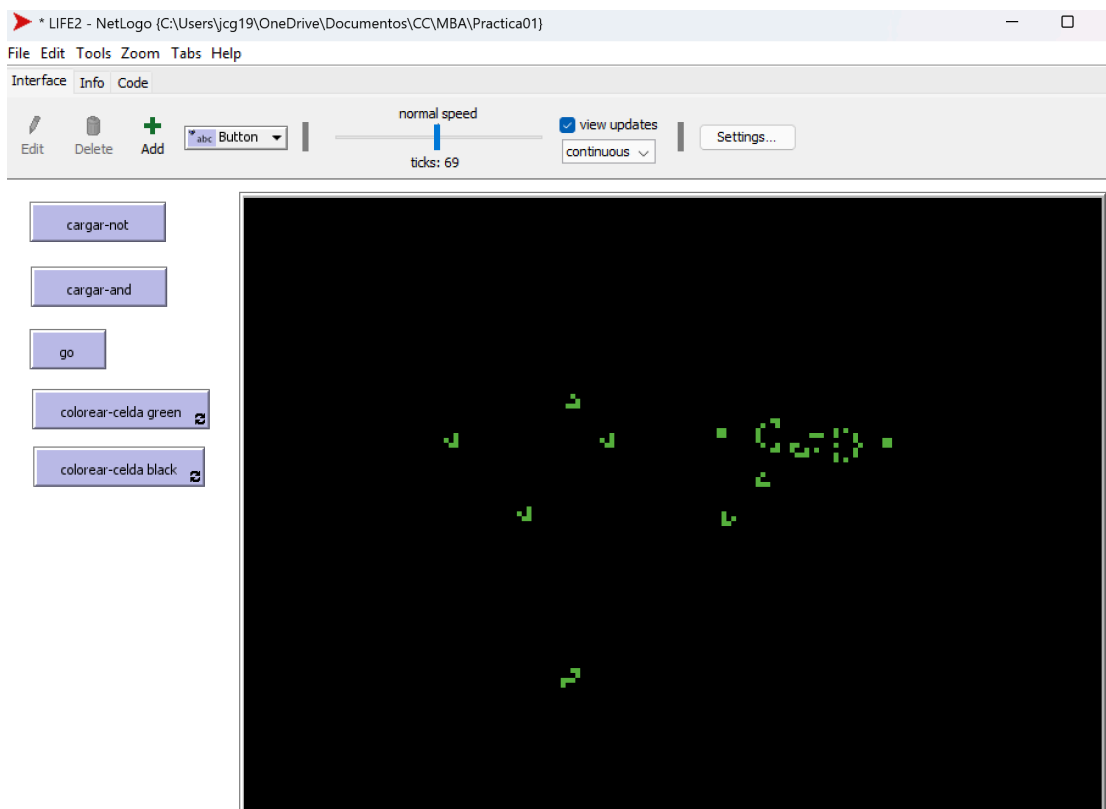


Para empezar la simulación se debe dar click en el botón de go.
 Un ejemplo de la simulación es este, donde empezamos con $A_1 = false$, $A_2 = false$, $A_3 = true$, $A_4 = true$, $B_1 = true$, $B_2 = false$, $B_3 = true$ y $B_4 = false$, por lo tanto obtendremos una salida de false, false, true y false.

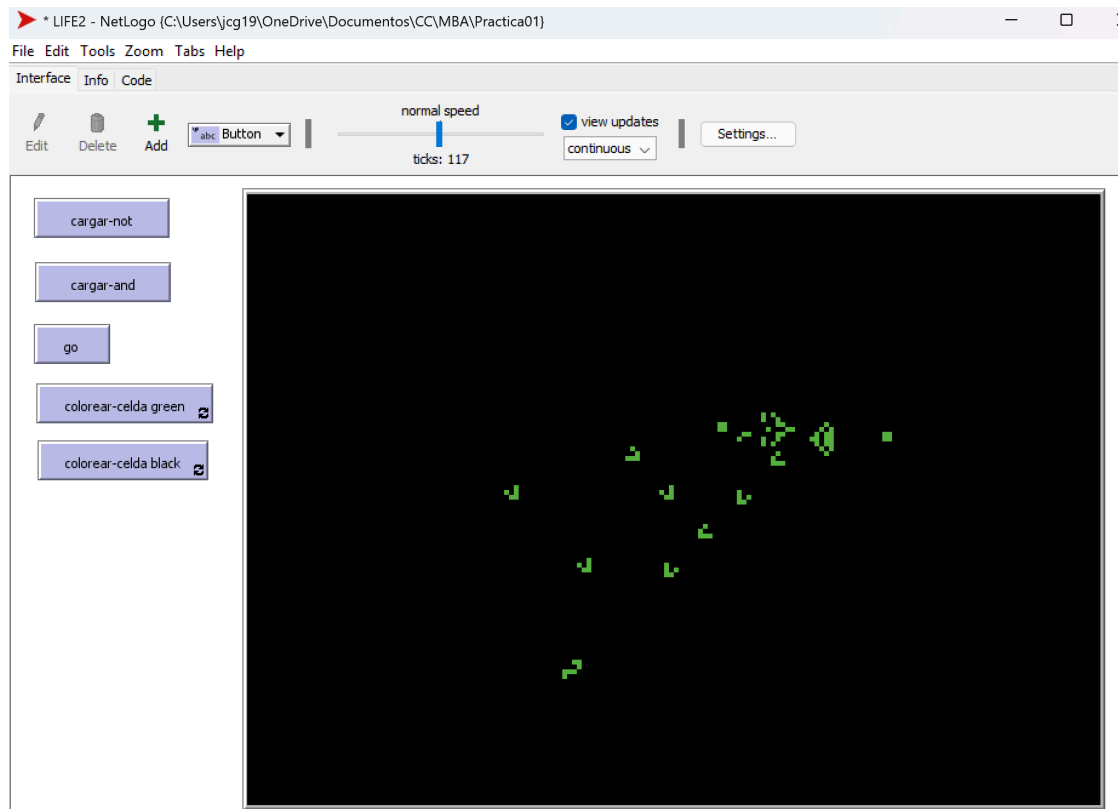


Vamos a ver como va progresando la ejecución.

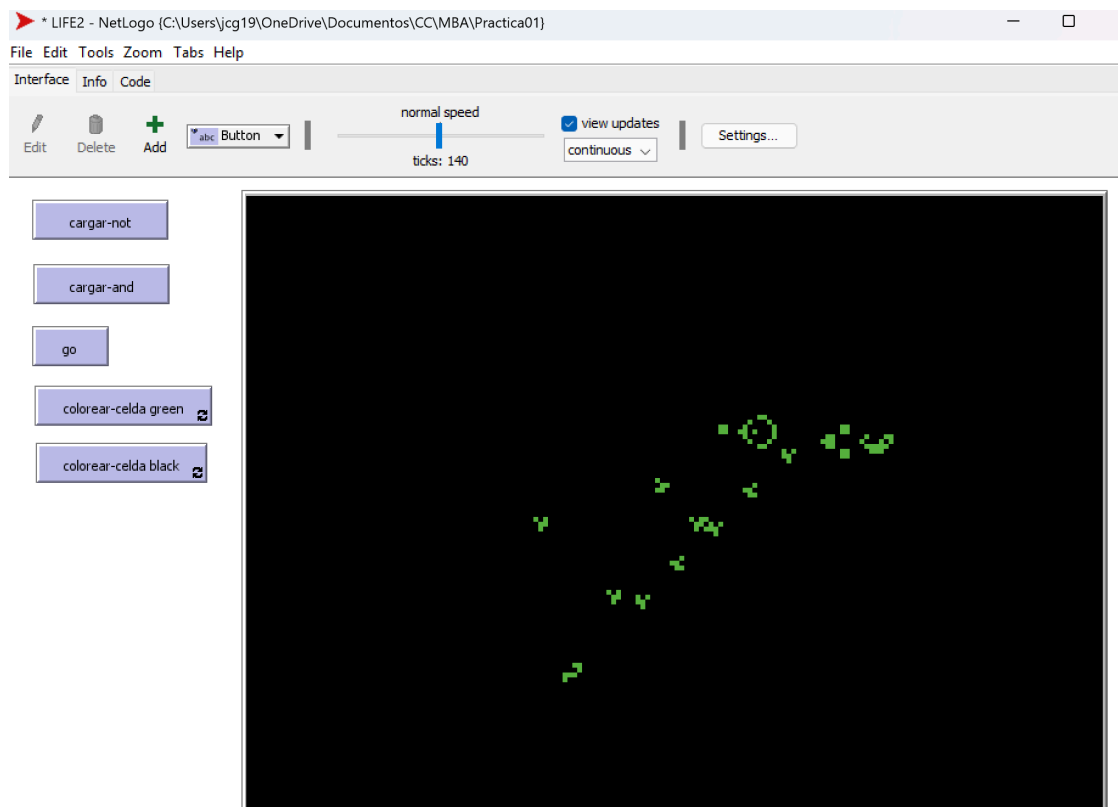
Lo primero que vamos a ver es que el primer glider generado por la pistola no es destruido por A_1 , ya que es false (no existe), por lo tanto va seguir avanzando.



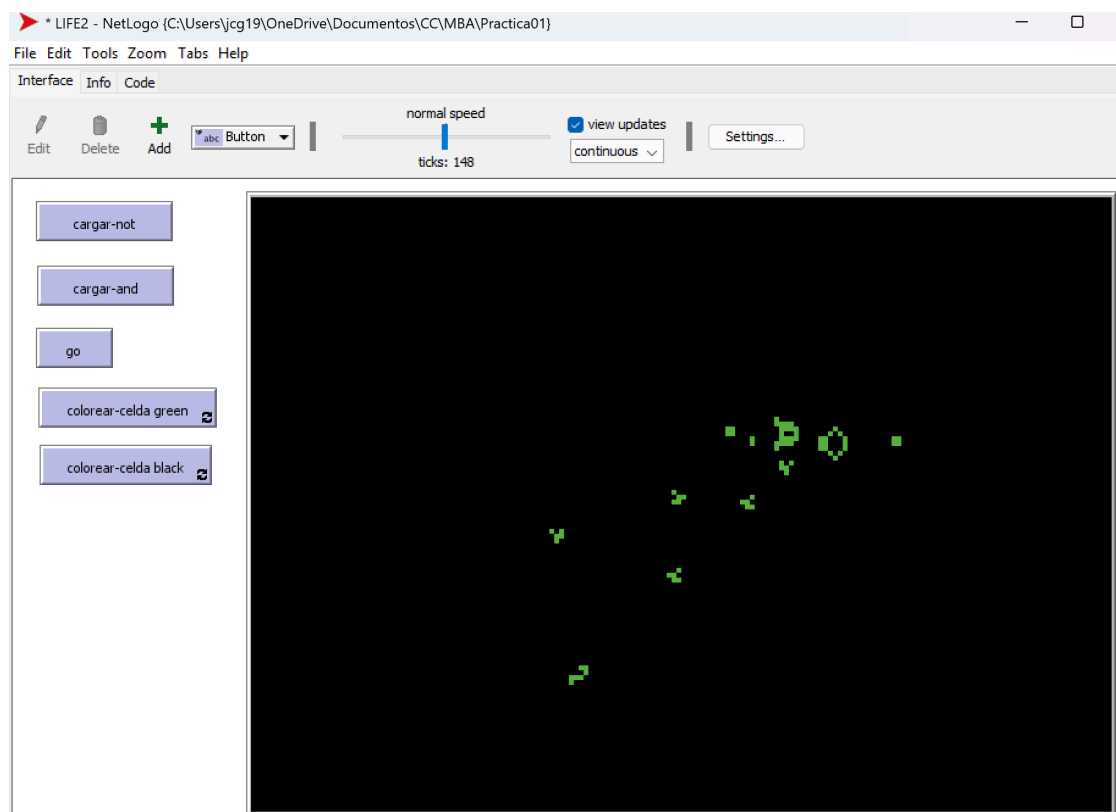
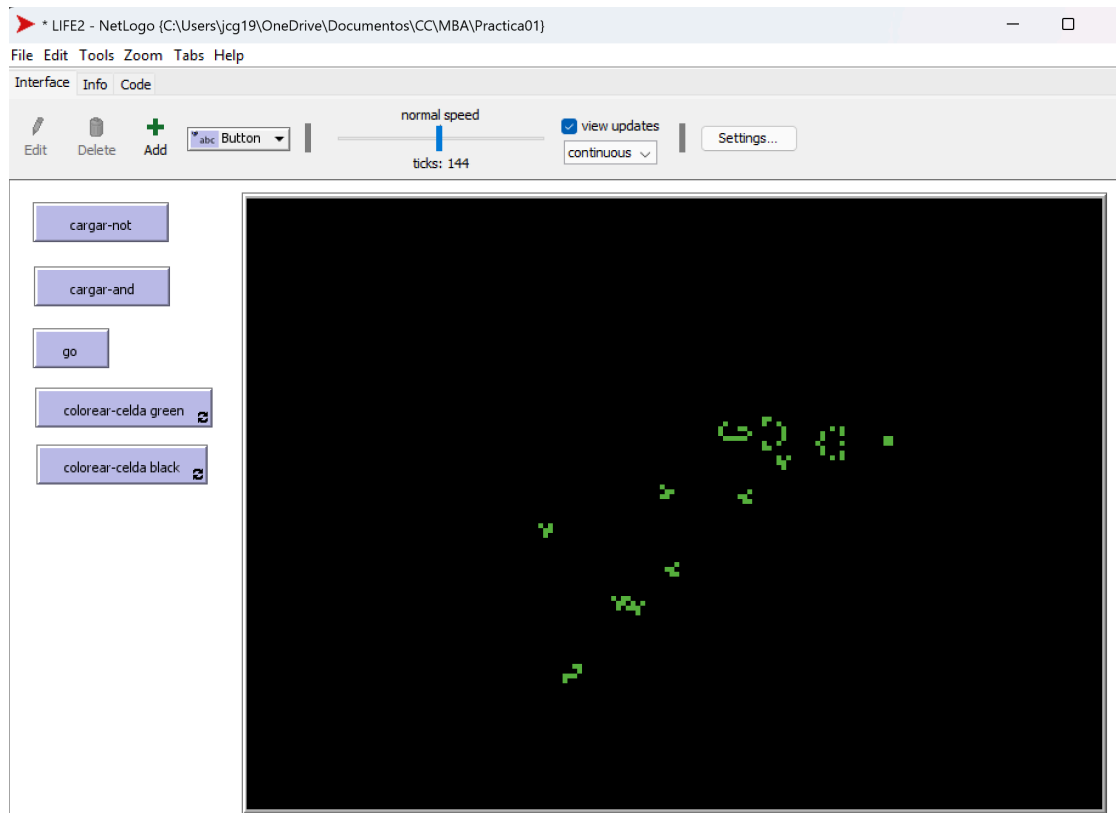
Después observamos que el segundo glider generado por la pistola tampoco es destruido por A_2 , ya que es false (no existe), por lo tanto va seguir avanzando.



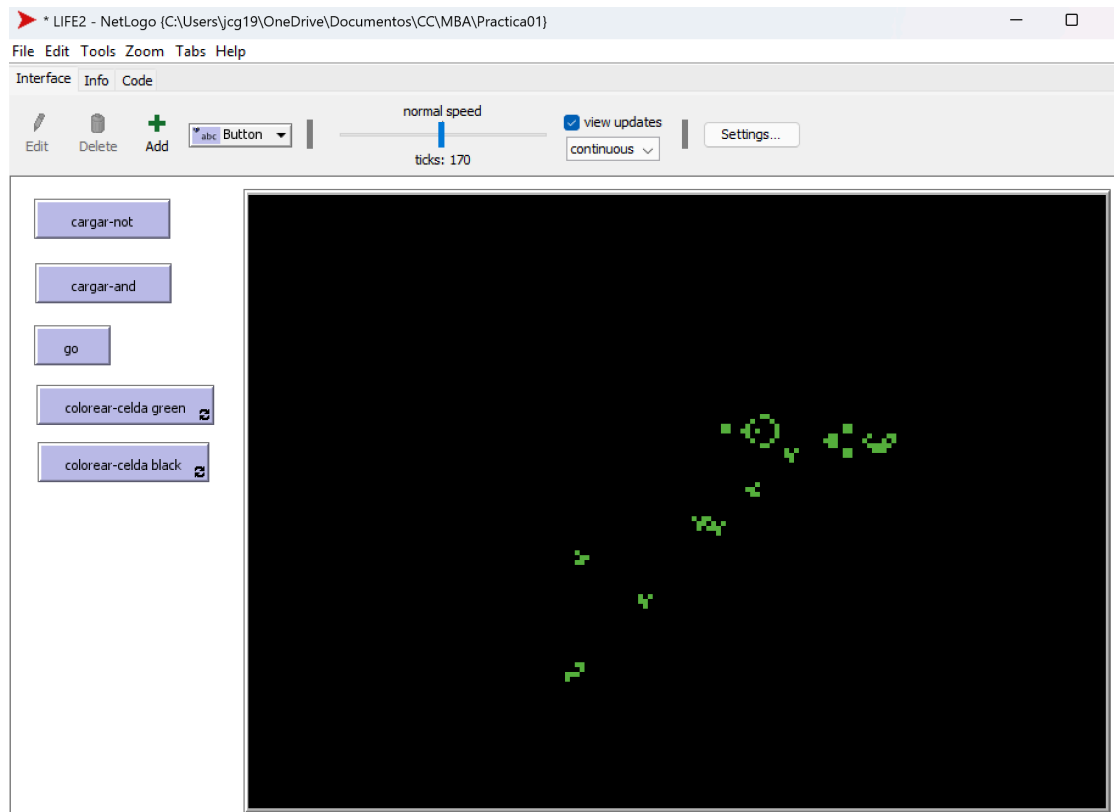
Luego vemos que el tercer glider generado por la pistola va a chocar con A_3 y se ven a destruir.



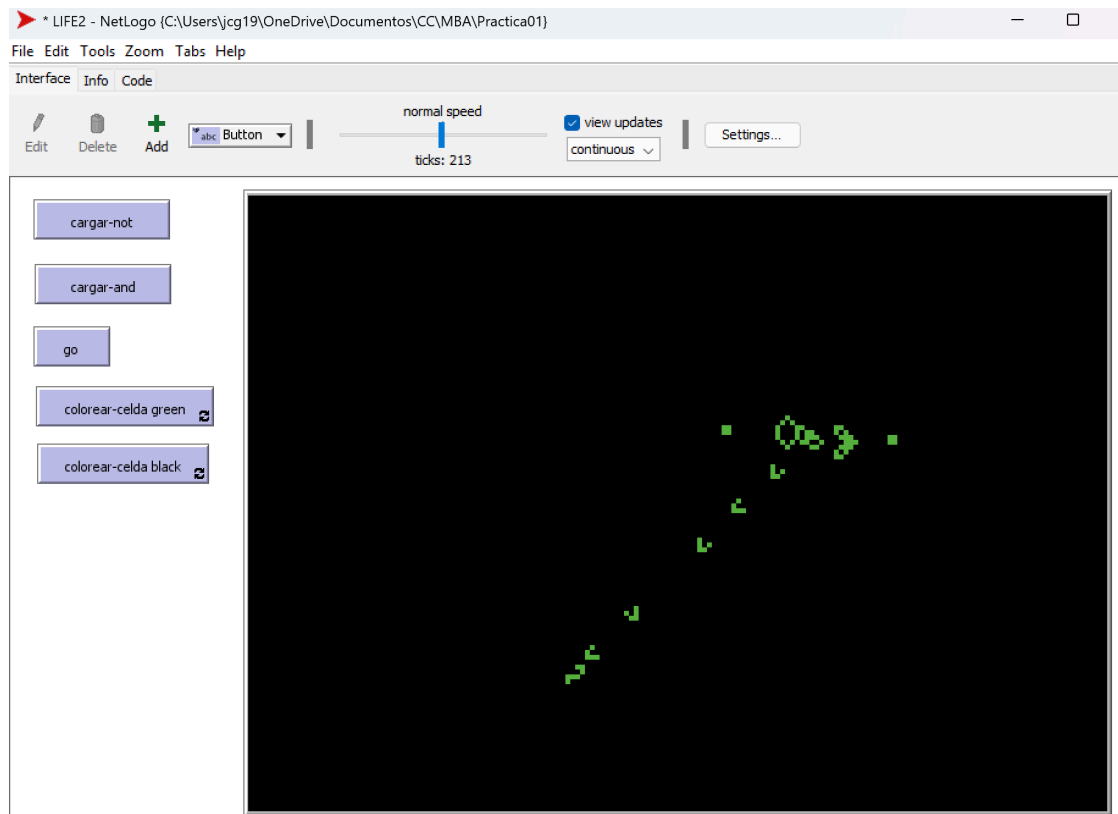
Posteriormente observamos que el glider B_1 va a chocar con el primer glider generado por la pistola, por lo tanto se ven a destruir, y de esta manera tendremos que el resultado de A_1 and B_1 va a ser false, debido a que B_1 va a ser destruido y no podrá llegar a la parte de abajo.



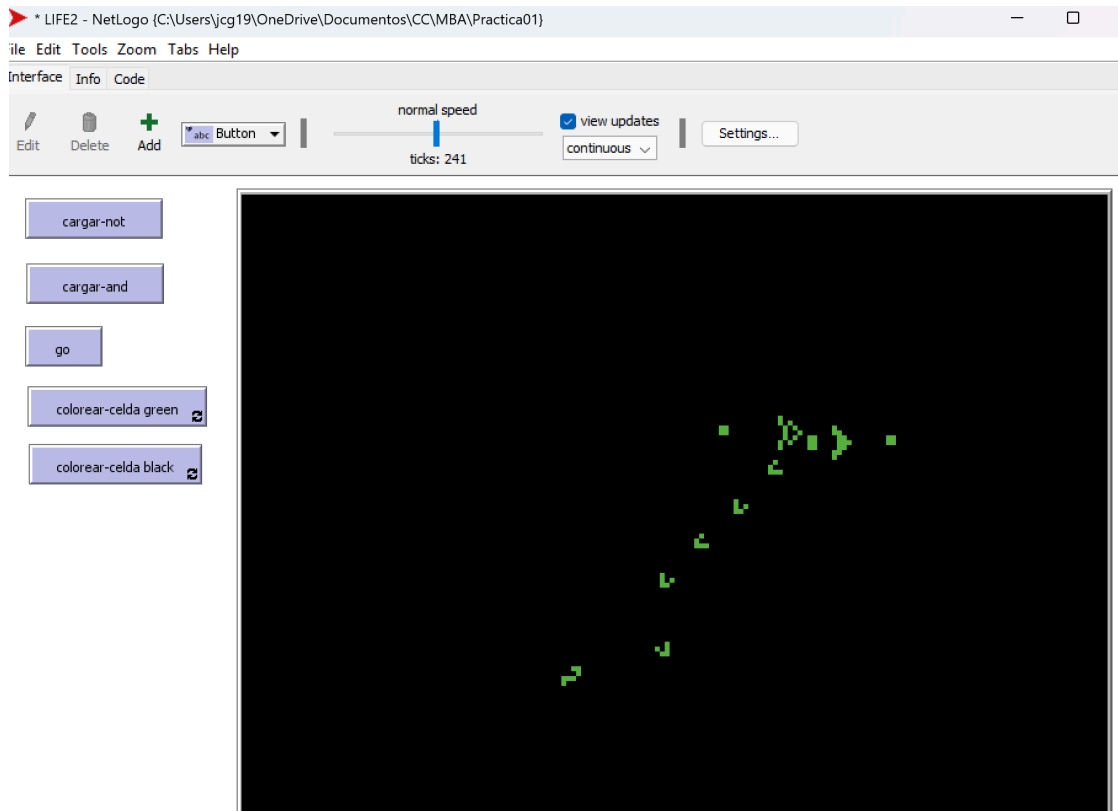
Después observamos que el glider A_4 va a chocar con el cuarto glider generado por la pistola, por lo tanto se ven a destruir. También vamos a notar que el segundo glider generado por la pistola no chocó con B_2 ya que no existe (es false) y por lo tanto va colisionar con el eater para ser destruido, y por lo tanto el resultado de A_2 and B_2 va a ser false, debido a que B_2 no podrá llegar a la parte de abajo.



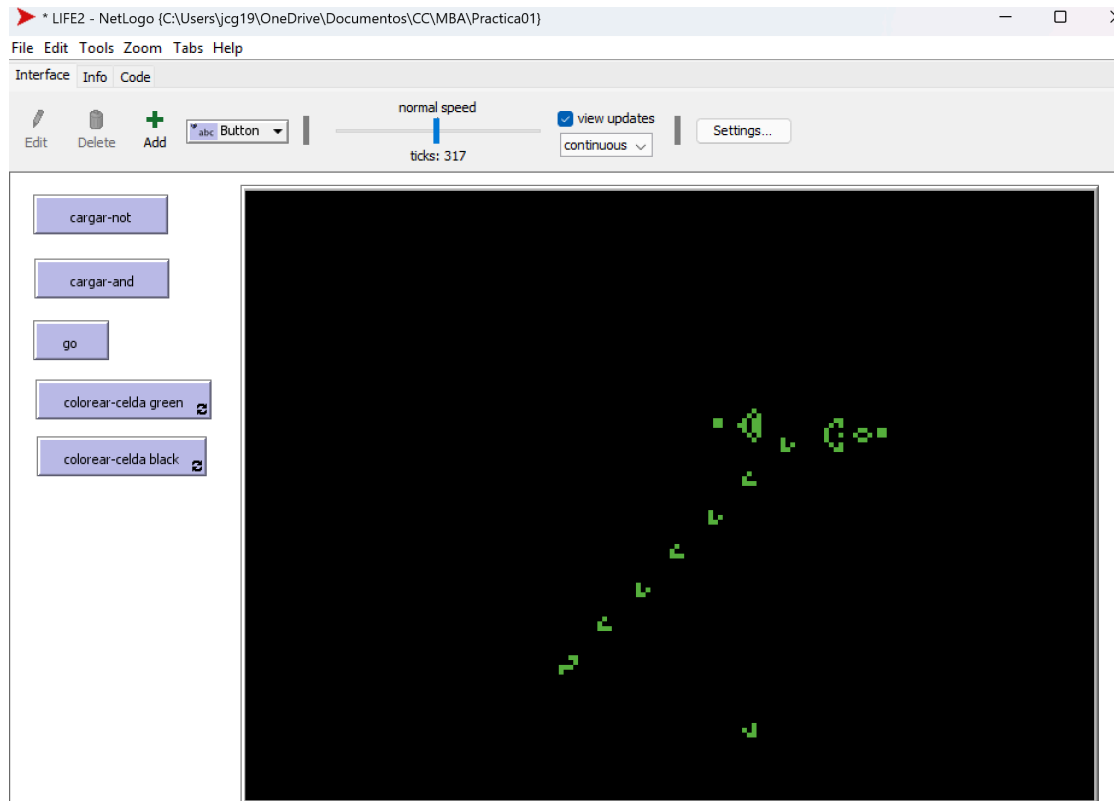
Luego vemos que el glider B_3 no va a chocar con el tercer glider generado por la pistola, ya que este fue destruido cuando chocó con A_3 , de esta manera tendremos que el resultado de A_3 and B_3 va a ser true, debido a que B_3 podrá llegar a la parte de abajo sin ser destruido, lo cual significa que es true. Otra cosa es que podemos ver como el segundo glider de la pistola es destruido por el eater.



Por último podemos observar que el cuarto glider de la pistola ya no existe debido a que chocó con A_4 pero como B_4 es false entonces B_4 no va a poder llegar a la parte de abajo, resultando en que el resultado de A_4 and B_4 va a ser false.



Podemos notar que estos resultados obtenidos es como funcionan las compuertas AND, ya que necesitamos que tanto A como B sean true, esto se logró con la pistola de gliders, ya que para que B llegue a la parte de abajo entonces A tiene que destruir el glider producido por la pistola, ya que si no lo destruye entonces choca con B impidiendo que llegue a la parte de abajo (como en el caso de A_1 and B_1).



Observación en la implementación de las compuertas AND y NOT se usa el archivo LIFE2, en este archivo no se puede modificar el tamaño del mundo (esto para que los métodos de cargar-not y cargar-and funcionen bien y no se queden partes de la compuerta fuera) y tampoco se pueden modificar las fronteras, son fijas (ya que si usamos fronteras periódicas entonces los gliders podrían destruir la compuerta).